

REMOTE SENSING IMAGE ACQUISITION, ANALYSIS AND APPLICATIONS

Module Two

Computer-based interpretation – fundamentals of machine learning

Voice-over script

John Richards
The University of New South Wales
The Australian National University

Lecture 1. Fundamentals of image analysis and machine learning

Slide 2.1.1 In the previous module we looked at how remote sensing imagery is acquired. We also looked at how the imagery is affected by atmospheric conditions and various geometric distortions.

We're now at the stage where we can concentrate on how to analyse the image data, in order to extract information of importance to operational remote sensing.

This module focuses on the range of machine learning methods that are used commonly for image interpretation in remote sensing. We will start with some simple, historical algorithms and move on to procedures that have gained popularity in the last two decades.

Slide 2.1.2 The material we are going to develop in this module requires an understanding of basic statistics, calculus, and vector and matrix analysis. If you do not have that background, the summaries should provide an overview of the important points. Also, we will step through the analysis slowly, and in detail, so that you should, at least, pick up the essence of what is being developed.

When we finish this module, you should be in the position to appreciate how a range of popular machine learning methods can be applied in remote sensing.

It is important to understand that, from an operational point of view, you will not have to engage yourself in the range of mathematics we are to encounter when applying the various algorithms in practice. Instead, commonly available software packages used for remote sensing purposes will contain the algorithms in the form that the user can employ readily.

Slide 2.1.3 By way of summary, recall that what we are doing is taking the multispectral or hyperspectral measurements recorded by a satellite or aircraft instrument and, via an appropriate machine learning technique, producing a map of labels for each of the image pixels. Those labels represent classes of ground cover.

In other words, we're doing a mapping from recorded data to a thematic map of labels.

Slide 2.1.4 There are several different classification scenarios that we will come across. Four are shown here.

In the first, we analyse pixels individually, based on their spectral measurements, and produce labels for them. Those labels should represent the ground covers in which we are interested. Sometimes these are called point classifiers, because they focus on individual points (pixels),

In the second, while we are still interested in labelling individual pixels, some algorithms allow us to do so by taking into account the possible labels on the surrounding pixels. That is called context classification, or more properly spatial context classification; it is based on the idea that neighbouring pixels are likely to be of the same ground cover type.

In the third case we show the need to identify pixels using a range of sensor and data types—here optical, radar and thermal. Algorithms for this so-called multisource problem are difficult to come by, although some authors ignore the intrinsic differences in the data types and simply concatenate them into a single vector, to which they then apply point classification techniques.

Finally, particularly with high spatial resolution imagery, we might be interested in identifying objects. Those objects might be buildings in urban landscapes, or aircraft and other vehicles in surveillance applications.

Most of our work in this module will focus on the first scenario, but we will, later on, engage with the concept of spatial context.

Slide 2.1.5 Remember from our earlier work classification, or machine learning in remote sensing consists of a number of stages. The first is training: here sets of known pixels are used to train the classification algorithm that we're going to use.

The second stage goes by a number of names. Most commonly it is referred to as the classification step in remote sensing. In the machine learning community, it is called generalisation. It can also be called labelling or allocation.

The third stage is a central one when developing a classification. We need to know how well the trained classifier works on real, unseen data. It is generally called the testing phase and involves sets of labelled pixels that the analyst has put aside in order to check the performance of the classifier. We'll have more to say about this later, especially in Module 3 where we will treat the need to test the accuracy of a thematic map in some detail.

- Slide 2.1.6** We are now going to embark upon the development of four common classifiers.
- The first is the maximum likelihood classifier, which was the main algorithm used in remote sensing for many decades. It is still highly usable, particularly when the number of wave bands is small. But it does have limitations in respect of hyperspectral data, unless methods are employed to reduce data dimensionality beforehand. We will look at some of those methods in Module 3.
- The second method we will consider is the minimum distance classifier, again a long-standing technique which is often useful in its own right but, more importantly, it provides a good basis upon which to develop the next two methods.
- The third procedure we will analyse is the support vector machine. It is a popular classification method for use with hyperspectral data.
- Finally, we will look at the neural network. While that pre-dates the support vector classifier, it has a later version called the convolutional neural network that has gained particular popularity in the last 5 to 10 years. It often goes by the title of deep learning, which we will explain during the treatment.
- Slide 2.1.7** It is important here that we differentiate between supervised and unsupervised learning. Supervised classification is that in which labelled training data is used to establish values for the unknowns in the classification algorithm.
- When training data is not available, unsupervised techniques can be used to discover the class structure of an image data set. We will look at unsupervised methods in the last week of this module. They are based on the data analysis process called clustering.
- Slide 2.1.8** Before we commence developing the classifiers it is important to distinguish between what we as users think of as classes in the landscape, and what machine learning algorithms see as classes in the data. The two may not be the same, although we do hope there is a close mapping between them.
- For example, we would expect that pixels recorded by an instrument will tend to group in regions in spectral space if they belong to areas on the ground that exhibit the same spectral response. In remote sensing such spectral groupings are called spectral classes, or sometimes data classes, since they are the natural groups or classes in the data.
- The set of classes in which the user is interested are called information classes. They have names like natural vegetation, water, forest, crops, and so. One would hope that they would align one-for-one with the spectral classes, but that is often not the case. Part of the skill of the analyst is to discover whether such a mapping exists or whether several spectral classes may be needed to represent each information class properly. As a simple example, a single crop class may have several constituent spectral classes because the crop will exhibit slightly different spectral responses depending on the soil types on which it is sown, the availability of ground water and whether any particular portion of a crop is in shadow.
- We will have more to say about the relationship between spectral and information classes when we come to Module 3 of this course. But unless we say otherwise, we will assume in this module that our information and spectral classes are the same.

Slide 2.1.9 Here we just summarise the important points raised in this introductory lecture.

Slide 2.1.10 The third question here has implications for the classification algorithms we are going to develop.

Lecture 2. The maximum likelihood classifier

Slide 2.2.1 In this lecture we are going to develop the maximum likelihood classifier. We will see that it has a couple of variants, but all tend to reduce to the same underlying principle.

Slide 2.2.2 It is of value to recall that we model the data in a spectral space which has axes aligned with each of the spectral measurements of the remote sensing instrument. Here we use a simple two-dimensional version, which will be sufficient for our later developments but, as result of the mathematics we employ, can be conceptually generalised to any number of dimensions.

Importantly, we note that pixels of a given ground cover type tend to congregate together in regions of the spectral space. Although we admit the possibility of several spectral classes per information class, for simplicity in our development we will, as we said earlier, assume that each information class can be represented by a single spectral class.

Here we have shown a single spectral reflectance curve for each cover type, such that there is a single point per class in the spectral space. In practice, of course, variations will exist as shown in the next slide.

Slide 2.2.3 The natural variations in each particular cover type will exhibit as clusters of points in the spectral domain as seen here. The locations of the classes are consistent with the spectral reflectance data and tend to group around a point indicative of the mean of the relevant spectral reflectance curve.

Slide 2.2.4 The starting assumption for the maximum likelihood classifier is that the density of the cluster of the pixels of a given cover type in the spectral space can be described by a multi-dimensional normal distribution.

While this may seem a restrictive assumption, the normal distribution is convenient because it, and its properties, are well understood and, even if the actual density distribution is not normal, the normal model nevertheless works well. Also, later we will represent a given class by a set of normal distributions, which helps in overcoming many concerns with using this model.

Slide 2.2.5 What are the parameters of a normal distribution? They are the mean position and standard deviation in one dimension. Generalising to multiple dimensions, the parameters are the mean vector and the covariance matrix. The mean vector describes the location of the density maximum (or the centre of the spectral class) in spectral space, and the covariance matrix describes the spread in pixel density about the mean position. The covariance matrix is the multi-dimensional version of the variance of a one-dimensional distribution.

The formulas for the single and multi-dimensional distributions are seen to have a similar structure, as is to be expected.

Slide 2.2.6 We now focus the formula for the multidimensional normal distribution onto a particular class—class i .

We indicate that by adding the subscript i to the mean vector and covariance matrix, and, further, write the probability itself as conditional on the fact that we are looking at class ω_i .

We will adopt that nomenclature throughout this series of lectures. The class of interest will be indicated as ω_i . The probability of a pixel existing at position $\mathbf{x}$ in the multispectral space will be indicated as the conditional probability shown on the left-hand side of the equality here, and the class parameters $\mathbf{m}_i$ and $\mathbf{C}_i$, sometimes collectively called the class signature, will have subscripts indicating the particular class.

Note that if we have training data available for that class then we can estimate values for the components of the mean vector and the covariance matrix.

Slide 2.2.7 We now assume a normal probability distribution for each class. Using training data, we estimate the means and covariances of the classes. Once trained, we can then choose to allocate an unknown pixel with measurement vector $\mathbf{x}$ to the class of highest probability. Note that we are explicitly representing the probabilities as class-conditional probabilities. Because we are choosing the class of the highest likelihood (probability) this approach is called maximum likelihood classification.

In the diagram note how the contours of equal probability segment the spectral space into class regions.

There is however a better, more general approach...

Slide 2.2.8 Here we make an interesting distinction in conditional probabilities. At the top we show the class conditional probabilities. The set of conditionals tell us the probabilities of finding a pixel at spectral location $\mathbf{x}$ —i.e. with the measurement vector $\mathbf{x}$ —from each class.

Consider now the second set of probabilities on this slide. That expression says that, given we are interested in pixels at position $\mathbf{x}$ in spectral space, what is the probability that the corresponding class is ω_i ? This is a much better measure to use than the class conditional probability since it focusses on the correct class for the pixel rather than the likelihood that there is a pixel from a given class at position $\mathbf{x}$ in spectral space.

That gives us another way of classifying unknown pixels...

Slide 2.2.9 If we knew the full set of these new probabilities, then we could classify an unknown pixel with spectral measurement vector $\mathbf{x}$ by allocating it to the class of the largest of the set. We express that in the form of a decision rule as shown symbolically on this slide.

The new probabilities are called posterior probabilities, the reason for which will become clear soon.

The problem is we don't know the posterior probabilities, only the class conditional probabilities which we have estimated from training on labelled pixel data.

Slide 2.2.10 Bayes rule gives us a bridge between the two types of conditional probability. It introduces two new probabilities— $p(\omega_i)$ and $p(\mathbf{x})$. The latter is just the probability of finding any pixel with measurement vector $\mathbf{x}$. When we use Bayes theorem in the decision rule of the previous slide, we note that the probability $p(\mathbf{x})$ is common to both sides and can be dropped out. That leaves the rule shown in the second equation.

But what is the new probability $p(\omega_i)$?

Slide 2.2.11 The probability $p(\omega_i)$ is the likelihood of finding a pixel from class ω_i **anywhere** in the image—i.e. it is not $\mathbf{x}$ dependent. It is a property of the scene itself, and is called the **prior** probability, because it is the probability with which we can guess the class membership of an unknown pixel without the benefit of the remote sensing measurements. For example, if we knew the rough proportions of the classes, we could use them to guess the priors based on areas.

In contrast the $p(\omega_i|\mathbf{x})$ are called posterior probabilities because they are the probabilities with which we assess the class membership of a pixel **after** we have carried out our analysis using the information provided by the measurement vector $\mathbf{x}$ for the pixel.

If we had no idea of the priors we could assume that they were all equal, in which case the new form of the decision rule of the previous slide reverts to the one we started with, in which the decision is made using class conditional probabilities—the distribution functions—rather than posterior probabilities.

Slide 2.2.12 Remember the normal distribution is also called the Gaussian distribution. We will use that alternative from time to time.

Slide 2.2.13 For the last question note that if two events occur together—jointly—then the probability of the joint event is not order dependent. The joint probabilities are the same: $p(\mathbf{x}, \omega_i) = p(\omega_i, \mathbf{x})$

Lecture 3. The maximum likelihood classifier: discriminant function and example

- Slide 2.3.1** In this lecture we will develop some further aspects of the maximum likelihood classifier and provide an example. This is a slightly longer lecture because we have kept the material together in a related group of topics.
- Slide 2.3.2** In this slide we take the decision rule we have been using up to date and now substitute in that expression the formula for a multi-dimensional normal distribution. As we will see, some simplifications are possible that mean that we do not have to evaluate the normal distribution itself on each occasion when we wish to label a pixel.
- To make the result simpler we take the natural logarithm of the product of the class conditional and prior probabilities. As is well-known, the result is the sum of logarithms. And since the normal distribution contains an exponential term, taking the natural logarithm means the exponent is removed, as we see in the last three steps here.
- We have decided to name the logarithm of the product of probabilities as the **discriminant function** for reasons which will become clear shortly.
- Slide 2.3.3** When we examine the form of the discriminant function, we see that the first term contains no information that will contribute to discriminating among the classes. Therefore, we remove that term to give the slightly simpler version shown in the centre of this slide.
- We now express the decision rule in terms of the discriminant functions, which is the form used in remote sensing image analysis software.
- Slide 2.3.4** Now consider a simple example. We met this one briefly in the introductory material in Module 1. The image segment is a portion of a Landsat multispectral scanner image recorded in 1979. There are four obvious ground cover classes: vegetation, burnt vegetation which we will label as fire burn, urban and water.
- Slide 2.3.5** The first task in a supervised classification is to select training data—that is, pixels whose ground cover labels we know. In practice that may be an expensive and significant exercise, sometimes requiring site visits, the use of reference maps, air photos and the like. But in this case, we can see the ground cover types easily in the image, so that we can determine a set of training pixels by inspection.
- The numbers of training pixels per class are shown in the table.
- Slide 2.3.6** When the training data is presented to the classification software the first output is the set of class signatures: that is, the mean vectors and covariance matrices for the classes, as shown in the table on this slide. As we noted in the first module, the covariance matrices are symmetric.

Slide 2.3.7 Once the classifier has been trained it can be used to label all the pixels in the image as shown in the thematic map of this slide. Not only is a map produced, but we also have a table of the numbers of pixels per class which we can translate into areas, shown here in hectares.

Note that the training pixels represent only 7.5% of the scene. That is the benefit of supervised classification; by putting effort into labelling a small number of pixels ourselves through training, we gain the advantage of having the classifier label a much greater number for us.

Slide 2.3.8 One of the benefits of classifiers such as the maximum likelihood rule is that the mean vector elements represent the average spectral reflectance curves of the cover types, as seen here. In those curves we can identify many of the important properties of the scene, such as the loss of green biomass through the fire, and the fact that the urban zone is a mixture of bare surfaces and vegetation.

Slide 2.3.9 An important consideration when using any supervised classification procedure is to know how many training pixels per class are needed.

For an N dimensional space, the covariance matrix has $N(N+1)/2$ distinct elements. To avoid the matrix being singular at least $N(N+1)$ independent training samples are needed. Fortunately, each N dimensional pixel vector contains N separate samples in its set of spectral measurements. Therefore, a minimum of $(N+1)$ independent training pixels is required.

We usually try to obtain many more than that, so that we can assume independence and get good, reliable estimates of the elements of the mean vector and covariance matrix. With the maximum likelihood classifier, experience has shown that we should look for a bare minimum of $10N$ training pixels per class, but desirably at least $100N$ training pixels per spectral class, with more than that if possible.

Slide 2.3.10 For data with low dimensionality, such as multispectral images, the minimum of 10 - $100N$ pixels per class is usually easily achieved. But when we have to work with hyperspectral images, there can often be a problem with not obtaining enough independent training samples.

For example, if there are 220 wave bands we should be looking for about $20,000$ labelled pixels per class for good training. Clearly that is often a difficult target to meet. If we wish to use a maximum likelihood classifier with hyperspectral data, therefore, we have to use so-called feature reduction techniques to lower the data dimensionality beforehand, or else resort to other classifier methods that don't require as many samples for reliable training. We will look at those procedures soon.

Slide 2.3.11 This slide illustrates the point of not having enough training samples per class with increasing data dimensionality. The example is from an old paper but nevertheless illustrates the problem quite well. It involves an exercise with 400 training pixels per class over five classes. As the number of bands or features increases there is an improvement in classifier performance (or generalisation), but beyond about 4 features, or dimensions, the performance drops because of the poor estimates of the elements of covariance matrix. Note that for 5 features we should be looking for at least 500 pixels per class if possible.

This problem has been known in remote sensing for many years and goes under the name of the Hughes phenomenon. Is also an example of the curse of dimensionality often referred to in the machine learning literature.

Slide 2.3.12 When we commenced our discussion of classifier methods, we sketched the positions of the pixels from given classes in the spectral domain. Effectively, what our classifiers do is place boundaries between the classes. Pixels from one class lie one side of a boundary, while pixels from another class lie on the other side. Intuitively, one would expect that simple geometric boundaries would be less successful in separating two classes of pixel than boundaries of a higher order.

The way we find these boundaries for any classification algorithm is to find the locus of points in the spectral space for which the two relevant discriminant functions are equal.

By doing so, this slide shows that the locus of points for the maximum likelihood classifier is quadratic. In other words, the boundaries between classes are high dimensional circles, parabolas, ellipsoids, etc.

If you go right back to one of slides in the last lecture when we started our work on the maximum likelihood classifier you will see those sorts of boundaries—or decision surfaces—in the diagram showing three intersecting two-dimensional normal distributions.

Slide 2.3.13 Here we illustrate the quadratic nature of the surface between two classes on the left, and on the right the rather more complicated decision surfaces that can be obtained if we allow more than one spectral class per information class, which we will treat later.

Slide 2.3.14 The last two points are particularly important to a lot of what is to follow.

Slide 2.3.15 Some of these questions ask you to think about the shapes of the decision boundaries for the maximum likelihood rule under different conditions.

Lecture 4. The minimum distance classifier, background material

- Slide 2.4.1** We now commence a journey towards the development of more complex classifiers. To do so we are going to look at another very simple algorithm that underpins our further development. This is called the minimum distance classifier. It is even simpler than the maximum likelihood rule.
- Slide 2.4.2** Consider two classes of data which are linearly separable—i.e. they can be separated by a linear surface, or a straight line in two dimensions. If we knew the equation of that line, we could determine the class membership for an unknown pixel by seeing on which side of the line its spectral measures lie. How can we express that mathematically?
- Slide 2.4.3** The equation of a straight line is pretty simple in two dimensions, as shown here. It is helpful to write it in the generalised form shown since that allows it to be taken to any number of dimensions as seen on the bottom of the slide.
Incidentally, in more than two dimensions we refer to the linear surface as a hyperplane.
- Slide 2.4.4** Here we write the equation in vector form, which is compact and allows manipulation by the rules of vector algebra, when needed. Note that we can use either the transpose expression or that using dot products—both are equivalent versions of the scalar product.
When we use the equation of the hyperplane in classifier theory, we often refer to the vector of coefficients w_i as a weight vector. Usually w_{N+1} is not included in the weight vector and instead is sometimes called the offset.
- Slide 2.4.5** Having expressed the hyperplane in vector form we now have an elegant expression for the decision rule to apply in the case of a linear classifier. The rule evaluates the polynomial for a given value of the measurement vector. If it is positive, then the corresponding pixel lies to the “left” of the hyperplane and thus is labelled as coming from class 1. If it is negative, then the pixel is from class 2.
This decision rule will feature often in our later work and will be the basis of further developments.
- Slide 2.4.6** How do we find the hyperplane? That requires finding values for the weights and offset.
As with all supervised classification methods that entails using sets of training pixels. We take that further in the next lecture.
- Slide 2.4.7** The linear classifier is the simplest of all. It is important that you understand it fully in order to appreciate later developments. Take particular note of the equation of a hyperplane (a multi-dimensional linear surface).
- Slide 2.4.8** Again, you need to be happy you understand these concepts in order to appreciate what is to follow.

Lecture 5. Training a linear classifier

- Slide 2.5.1** In this lecture we are going to look at training methods for linear classifiers, and in particular the minimum distance method. We will then use that material to help us understand and develop training methods for more complex classification techniques.
- Slide 2.5.2** Remember we are assuming at this stage that the two classes of pixel that we are dealing with are linearly separable—that is, a straight line can be placed between them. Many data sets however are not linearly separable. We will meet some later but recall for the moment that the maximum likelihood classifier is able to separate data sets with at least quadratic hypersurfaces.
- Slide 2.5.3** There are many acceptable linear decision surfaces that can be placed between two linearly separable classes, as illustrated here. One of the earliest methods for training, which goes back to the 1960s, involves choosing an arbitrary linear surface. That choice will almost certainly not be in the right position in that it will not have the classes on the right sides of the hyperplane. But then, by repeated reference to each training pixel in turn, the hyperplane is gradually iterated into an acceptable position. The book by Nilsson referenced here shows that method.
Note a restriction here—we are only dealing with two data classes. We will have to embed this method into some form of multi-class process later on.
- Slide 2.5.4** A better approach might be to choose as the separating hyperplane that which is the perpendicular bisector of the line which joins the means of the classes, as shown here. We can find that line as the locus of the points that are equidistant from the two class means. Note that we use the nomenclature $d(\mathbf{x}, \mathbf{m}_i)$ to represent the distance vector between two points.
- Slide 2.5.5** If the distances are equal then so will be their squares, saving the need to compute the expensive square root operation in software. So, we equate the squares of the two distances from a position $\mathbf{x}$ to the class means, leading to the equation of the linear surface at the bottom of this slide. Note that it has the same structural form as the equation of a straight line that we are familiar within two dimensions. Note that we have had to use two vector identities in coming to this result.
- Slide 2.5.6** Although we have computed the equation of the decision surface, in the minimum distance rule we actually don't compute the hyperplane explicitly. Instead, to label an unknown pixel we just compare the distance squared to the class means and allocate the pixel to the class of the closest mean. That suggests that we can actually account for as many classes as we need to. In this slide we have shown three classes and given a general decision rule for the minimum distance classifier.

Slide 2.5.7 So, in summary for the minimum distance classifier:

1. Training data is used to estimate the class means. Fewer pixels are needed compared with the maximum likelihood classifier since no covariance matrix estimation is required.
2. Unknown pixels are allocated to (labelled as) the class of the closest mean.
3. It is a linear classifier
4. It is a multi-class technique
5. It is fast in both training and classification

Slide 2.5.8 We are now at the stage where we can look in summary at the two classifiers we have treated so far and see the steps the user follows in practice. Although in our lectures we have looked at the mathematics of the techniques, you do not need to know that material in practice, although it does help you understand how the algorithms work and their comparative limitations.

Note that three of the steps are common to both approaches (highlighted in blue) and will also be common to any of the classifiers we look at in this course: they are training, thematic mapping and accuracy assessment.

The approaches differ only in how the classifiers are trained and how they are applied to unseen pixel data. If class conditional probabilities are used with the maximum likelihood method, there is an assumption that prior probabilities are available. Again, the software looks after that step, provided the user can assign values to the priors.

Note particularly the last step. Rarely, in a real classification task, will the accuracy on all classes be acceptable the first time around. Instead, the analyst may have to consider whether some classes have been missed, leading to their pixels being mis-allocated to another class, or whether some classes are too broad spectrally and should perhaps be sub-divided into constituent sub-classes (or spectral classes). Again, we will have more to say about this when we look at classification methodologies in module 3.

Slide 2.5.9 We are going to meet a number, but not all, classifiers that are used regularly in remote sensing. Some are quite complex and require a lot of user effort to make them work effectively, but they can give exceptionally good results. When selecting a method, though, it is important not to go overboard in always choosing the newest and potentially the most complex one. Often the simple algorithms will work just as effectively in many real-world applications.

Slide 2.5.10 Remember what we said at the start of this course. While we are necessarily spending a lot of time developing classifier algorithms, our course objectives are remote sensing and its applications. So, we need to keep that in mind when evaluating classifier methods. Ultimately, we have to embed them into a methodology, which we will do in module 3.

Slide 2.5.11 The first two questions here just test your knowledge of vector algebra, while the last two set you up for what is to come.

Lecture 6. The support vector machine: training

- Slide 2.6.1** We are now going to build on the work we did with the simple linear classifier to develop the support vector machine, a technique that became very popular about 15-20 years ago and is still regarded by some as the benchmark classifier for hyperspectral imagery.
- For those who come from a remote sensing background, rather than from the machine learning community, following the full development of the SVM can be challenging. For the same reason, we will not cover the full range of SVM theory in these lectures, but instead present just sufficient of the material so that the important points can be appreciated.
- For those interested in a fuller theoretical treatment, although from a generalised machine learning and not a remote sensing perspective, see C.M. Bishop, *Pattern Recognition and Machine Learning*, Springer, N.Y., 2006.
- Slide 2.6.2** The support vector classifier has a similar objective to the minimum distance classifier in that it is trying to find a hyperplane which optimally separates two classes of data, but its approach is different. In the SVM we look for the hyperplane which is mid-way between the two classes. We define it in terms of two marginal hyperplanes that just touch each of the classes, as seen in this diagram.
- Slide 2.6.3** There are several stages to the full development of the support vector classifier. It is helpful to review them at the outset so we know the direction we are taking in the overall development because the journey through the theory can be a bit tedious and, if we are not sure what our objectives are, it is easy to lose our way.
- The first two steps are shown here: one is to find the optimal decision surface, on the assumption that the classes are linearly separable; the second is to accept the fact that most data sets will not be perfectly separable, and that we will have to adjust our training process to accommodate the fact that there will be some class overlap, as seen on the right hand side diagram.
- Slide 2.6.4** The third step in our development of the SVM will be to make an adjustment for data that is not linearly separable. We will then move on to the final stage, which is to accommodate multiclass, as against binary, data sets. The fundamental SVM algorithm is just binary, so a multiclass strategy is required if it is to work with remote sensing problems that involve several data classes.
- Slide 2.6.5** Let's now look at the first step—finding the optimal position of the decision hyperplane. Again, it has the same mathematical form as the hyperplane for the minimum distance classifier. It is helpful if we can make the equations of the marginal hyperplanes take the forms shown on the diagram in blue; that is because we can scale the parameters in the decision rule, involving adjusting the offset.

Slide 2.6.6 We now need an objective that will lead us to the best hyperplane. We adopt the goal of finding the hyperplane which is mid-way between the two marginal hyperplanes which are furthest apart—that is, that have the largest margin between them.

From vector algebra we can show that the margin is given by the expression on this slide, where $\|\mathbf{w}\|$ is the “size” of the weight vector—the Euclidean norm, which is the square root of the sum of the squares of the vector elements, as is well known. We do that analysis on the next slide.

Slide 2.6.7 The margin is derived by taking the difference in the two perpendicular distances from the origin to the marginal hyperplanes. Note that when we apply that formula to the marginal hyperplanes the “1” on the right-hand side of the equals sign has to be taken into account as part of the offset or intercept, as seen in the second and third equations.

Slide 2.6.8 Thus, in seeking to maximise the margin we want to minimise the norm of the weight vector. However, unless we constrain that objective, we could make the margin as large as we like, but that will undoubtedly cause some of the pixel vectors to fall on the wrong side of their respective marginal hyperplane. So, we have to introduce a constraint to make sure that that does not happen. Such a constrained minimization (or optimization) can be carried out by the process called Lagrange multipliers, in which we set up a Lagrangian function as in the next slide.

Slide 2.6.9 The Lagrangian function that we wish to minimize is

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i f_i$$

where the $\alpha_i \geq 0$ are a set of parameters called the Lagrange multipliers, and the f_i are conditions, one for each training pixel, that ensure that the pixels are on the correct side of their marginal hyperplane. We choose as the conditions that ensure all the pixels stay on the respective correct sides of their hyperplanes that

$$y_i(\mathbf{w}^T \mathbf{x}_i + w_{N+1}) \geq 1 \text{ or } y_i(\mathbf{w}^T \mathbf{x}_i + w_{N+1}) - 1 \geq 0$$

where the binary variables y_i are indicators of the actual class for the i^{th} training pixel, as shown in red in the middle of the slide.

Slide 2.6.10 Thus, the Lagrangian to be minimized is

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i \{y_i (\mathbf{w}^T \mathbf{x}_i + w_{N+1}) - 1\}.$$

We need to find the weights in the hyperplane expression that minimize this expression. While we are trying to do that, the second term in the Lagrangian is trying to push it up if pixels are on the wrong side of their respective hyperplane. So, the hyperplane needs to be placed such that that effectively doesn't happen. The mathematics now becomes a little tedious but leads to some remarkable and important results. If you choose not to follow the detail, we will still summarise the important results at the end.

Slide 2.6.11 To minimize the Lagrangian with respect to the weights we have to differentiate it, as seen here, making use of the fact that we can express the vector norm in the form of a dot product (i.e. $\mathbf{w}^T \mathbf{w}$). The result shows us that we can find the set of weights, provided we know the values of the non-negative Lagrange multipliers α_i . This tells us that the decision surface is found from the set of training pixels and their classes, as is usual in classifier training.

Slide 2.6.12 We also have to minimize the Lagrangian with respect to the offset w_{N+1} which we do on this slide. However, note that this gives us another interesting condition, namely $\sum_i \alpha_i y_i = 0$. Note also from our previous equation (C) we get the helpful formula for the square of the weight vector norm shown in the centre of this slide. Our previous Lagrangian formula of

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i \{y_i (\mathbf{w}^T \mathbf{x}_i + w_{N+1}) - 1\},$$

using (D) gives us the expression in (E).

We are now in the position to find the α_i . Remember, they are trying to make the Lagrangian large to keep their respective pixel vectors on the correct sides of the separating hyperplane, so we now maximise (E) with respect to those Lagrange multipliers—see the next slide.

Slide 2.6.13 This maximization is complicated, so it is normally carried out numerically, to generate the values of the α_i . However, there is another constraint we can use to help us simplify the situation. It is one of the KKT conditions and says $\alpha_i \{y_i (\mathbf{w}^T \mathbf{x}_i + w_{N+1}) - 1\} = 0$ which is quite extraordinary since it tells us that either $\alpha_i = 0$ or $y_i (\mathbf{w}^T \mathbf{x}_i + w_{N+1}) = 1$. This last expression is true only for pixels lying exactly on one of the marginal hyperplanes, in which case the corresponding Lagrange multipliers α_i are non-zero. For training pixels away from the marginal hyperplanes, the expression inside the curly brackets is non-zero so the constraint can only be satisfied if the corresponding α_i are zero. Those pixels are therefore not important to the training process—it is only those lying on the marginal hyperplanes. We call those **support (pixel) vectors**, since they are the ones that support training.

Slide 2.6.14 We now have values for the relevant α_i . Thus, we now know all the variables that define the weight vector and thus, once we have found w_{N+1} , we can define the decision surface—the central separating hyperplane.

Since the α_i are zero for all but the support vectors, we find that the weight vector expression is simplified to a sum over just the set of support vectors $\mathcal{S}$. Once we know the w_{N+1} the support vector classifier has been completely trained. We will see how to do that in the next lecture and look at the next steps needed to ensure the SVM is a practical classification method.

- Slide 2.6.15** The process of minimizing the norm of the weight vector (maximizing the margin between the two classes), constrained by ensuring that the training pixels remain on the correct side of their respective marginal hyperplane leads to the amazing, but probably logical result, that it is only those training pixel vectors that lie on the marginal hyperplanes that are important in defining the decision surface. They are the support vectors.
- Slide 2.6.16** In the last question you will need to use a bit of imagination to come up with a distribution of training pixels such that the perpendicular bisector of the line between the class means does not always separate classes which are linearly separable.

Lecture 7. The support vector machine: the classification step and overlapping data

- Slide 2.7.1** We now take the support vector classifier to the next stage of development by looking at the actual decision rule and how we can handle the practical case of overlapping classes.
- Slide 2.7.2** Here we summarise how decisions about the class membership of an unknown pixel with measurement vector $\mathbf{x}$ are determined using the support vector machine. Although the decision rule shown in the centre of the slide is general to all linear classifiers, in the case of the SVM it is made simpler because the weight vector (at the top) and thus the decision rule is specified just in terms of the support vectors—those that lie on the two marginal hyperplanes. Given that the Lagrange multipliers α_i (and the support vectors) have been found during training the decision rule is simple and fast to apply.
- Slide 2.7.3** In this slide we find the one remaining unknown—the offset w_{N+1} . It is most easily found by choosing a support vector from each class and substituting them into their respective marginal hyperplane equations. Given that we know the weight vector we can find the value of w_{N+1} . To get a more reliable estimate, sets of support vector pairs can be chosen and the set of values of w_{N+1} so generated can be averaged.
- Slide 2.7.4** We now have to address the next three steps; the problem with the SVM assuming the classes are completely separable linearly, the fact that it is only a linear classifier and the fact that it is a binary classifier. We now look at the first of those—handling overlapping classes.
- Slide 2.7.5** For real data it is highly unlikely that two classes will be linearly separated as in our previous analysis. Instead, there is likely to be overlap as shown in the diagram here. To make the support vector machine able to handle such realistic situations we need to introduce some slackness into the training process, by which we mean we cannot achieve a best separating hyperplane, but probably one that is as best as we can do.
- Slide 2.7.6** The approach taken is to introduce a set of “slack” variables ξ_i one for each training pixel. Their role is to allow a softer decision to be taken concerning the class membership of a pixel. Note the new version at the bottom of this slide.
- Slide 2.7.7** We choose values for the slack variables as shown on this slide. If they are zero, then the corresponding pixels are correctly located. If they are unity, then the pixel sits directly on the separating hyperplane. If they are greater than one, then the corresponding pixels are on the wrong sides of the separating hyperplane. Otherwise they lie on the correct sides of the separating hyperplane but in the gap between that decision surface and the correct marginal hyperplane.

Slide 2.7.8 Since the ξ_i are positive for pixels located in the wrong class or in the gap between the marginal hyperplanes and the decision surface, their sum is a good measure of the total error incurred by poorly located pixels during training.

What we want to do now is maximize the margin, as before, but also minimize the error caused by poorly located pixels. We have a decision to make. Is it more important to maximize the margin, or minimize the class-overlapping pixels? Or should we seek a compromise? We can seek a compromise by adding a proportion of the sum of the ξ_i to the norm of the weight vector as a new measure to be minimized, as seen in the equation at the bottom of this slide. However, that minimization has constraints on it ...

Slide 2.7.9 There are two constraints:

- Similar to the case of non-overlapping data we seek to ensure that the argument $y_i (\mathbf{w}^T \mathbf{x}_i + w_{N+1}) - 1 + \xi_i$ remains positive. (Previously this constraint did not contain the ξ_i).
- Secondly, we need to ensure that each ξ_i remains positive, as defined.

Again, using the process of Lagrange multipliers, that leads to the Lagrangian at the bottom of this slide, which we now seek to minimize.

Slide 2.7.10 As with the case for non-overlapping data a numerical solution is used to find the two sets of Lagrange multipliers. However, the user has to specify the value of the parameter C beforehand, usually called a regularization parameter. We will see how that is done when we come to the examples later. Once a value is given for C, the numerical solution produces the required set of α_i . The decision rule stays the same as before, but the α_i will now not be optimal for a maximum margin but will reflect the compromise between maximizing the margin and minimizing the error due to over-lapping training pixels.

Slide 2.7.11 Effectively, what we are trying to do with the slack variable approach is to recognize that there will be a decision surface which is the best choice in terms of minimizing classification error caused by overlapping pixels.

Slide 2.7.12 Would some form of trial and error approach be acceptable as an answer to the last question?

Lecture 8. The support vector machine: non-linear data

- Slide 2.8.1** In this lecture we look at the third stage of the development of the support vector machine. We introduce a transform that, in principle, changes a data set that is not linearly separable into one that is. We will see, surprisingly, that we don't actually have to know the transform itself!
- Slide 2.8.2** Let's examine critically the form of the decision rule. Note particularly that the pixel vector $\mathbf{x}$ doesn't actually appear on its own but rather is always in combination with the support vectors, in the form $\mathbf{x}_i^T \mathbf{x}$. Suppose we now apply an unspecified transformation to the data space—call it $\phi(\mathbf{x})$. Thus the product of the support vector and the unknown pixel vector will, in the transformed space, become $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}) \rightarrow k(\mathbf{x}_i, \mathbf{x})$. We call that transformed product a kernel and represent the result by $k(\mathbf{x}_i, \mathbf{x})$.
- Slide 2.8.3** Now insert that kernel function into the decision rule. All that we have really done here is transform the original pixel space, but we can interpret the resulting expression as one which says that we need to know the kernel but we don't actually need to know the transform $\phi(\mathbf{x})$ that led to it. The real question then becomes: what functions can we use as kernels? Because they represent a scalar product, they have to be decomposable into that form. Some common kernels that satisfy that condition are shown on the next slide.
- Slide 2.8.4** The most common kernels encountered in remote sensing applications are the last two shown here, although the polynomial could also be used. Note that they have parameters for which we need to choose a value. That is often done by running a series of trials to find which value works best, as we will see later.
- Slide 2.8.5** To see how kernels work, we will look at a very simple example. We will use the first example from the previous slide—a quadratic kernel—with a two-dimensional data space. First, we demonstrate that the chosen kernel can be decomposed into a scalar product, which we do by expanding it and then re-writing it as shown on the bottom half of this slide.
- Slide 2.8.6** Having decomposed it we can now see what the underlying transformation function is. Remember, we don't need to know this in practice. We are just looking at it here in the context of this simple example to see how kernels work. The transformation projects the data into a three-dimensional space of the squares and cross-products of the original data axes.

- Slide 2.8.7** We now apply the transformation to the data set below on the left. The two classes of data lie on either side of the quadrant of a circle, and clearly are not linearly separable. After transformation the data is linearly separable in a two-dimensional space. Even though a third dimension has been created in the transform of the previous slide, it is not needed.
- Slide 2.8.8** Take particular note of the second and fourth points here.
- Slide 2.8.9** Would some form of trial and error approach be acceptable as an answer to the first question?

Lecture 9. The support vector machine: multiple classes and the classification step

Slide 2.9.1 We now come to the fourth step in the SVM—how to turn a binary classifier into a multi-class process.

Slide 2.9.2 The classical way of turning a binary classifier into a multiclass machine is to embed it in a decision tree, the simplest of which is that shown in this slide. Here each class is separated from the remainder sequentially. It is immediately obvious though that a problem with this approach is that the training data sets used to generate each binary decision are very unbalanced, particularly near the top of the tree. Furthermore, there is probably an optimal order in which to do the class separations, but we don't know that beforehand.

Slide 2.9.3 A preferable approach is to use the one-against-all strategy in which a set of binary classifiers are trained in parallel. Each classifier separates one class from the rest, so there are as many classifiers as there are classes. Having been trained then, when used to classify unknown pixels, the decision rule applied at the output of the tree selects the most favoured label, based on the SVM decision rule.

Slide 2.9.4 Another approach is the one-against-one strategy. The topology is the same as in the one-against-all approach of the previous slide but here each classifier is trained to separate just two of the set of classes. All class pairs are implemented, so that $M(M-1)/2$ separate classifiers are needed. We get that number by looking at the number of times two classes can be selected from the available M .

The calculation is $\frac{M!}{(M-2)!2!}$.

Note that each class will appear $(M-1)$ times among the set of classifiers. An unknown pixel is placed into the class which has the greatest number of recommendations in favour of it among the $M(M-1)/2$ decisions. Because of the large number of individual classifiers involved, training can be quite time consuming.

Slide 2.9.5 We have now covered the four stages in the development of the support vector classifier, as above. We now wish to see how the machine can be used in practice, which we do by way of example in the next lecture.

- Slide 2.9.6** There a number of decisions that must be taken by the user before applying the SVM to a real remote sensing classification problem. Effectively, we have to choose the multiclass strategy to be used and the kernel. We then have to implement some process to find the best value for the kernel parameter and for the regularization parameter. In the next slide we put these steps into the overall classification methodology we outlined in the case of the maximum likelihood and minimum distance classifiers.
- Slide 2.9.7** We now examine the SVM from an operational strategy perspective. Recall we did that in lecture 7 for the maximum likelihood and minimum distance classifiers. The table here is identical to that earlier one but has some steps particularised to the SVM. The three steps which are common to all classifier methods are highlighted in blue.
- Slide 2.9.8** Please note that software is freely available for implementing a support vector machine. Some of the most common are shown here.
- Slide 2.9.9** The second dot point here again summarizes the four essential steps in the application of the support vector machine.
- Slide 2.9.10** These questions are designed to reinforce the important aspects of the chosen multi-class strategy for use with the support vector machine.

Lecture 10. The support vector machine: an example

- Slide 2.10.1** In this lecture we present two examples of the application of the support vector classifier in remote sensing
- Slide 2.10.2** We start with a simple example, using a segment of Quickbird imagery recorded in 2002. The image segment consists of 500x600 pixels, with four bands of data, as shown.
- Slide 2.10.3** Several cover types are evident in the image. Two of the information classes each consist of two spectral classes. The SVM was trained on the identified spectral classes.
- Slide 2.10.4** As required, the authors chose a set of training pixels (as fields of pixels), and a separate set of pixels to be used for testing the accuracy of the result of the classification.
- Slide 2.10.5** Here we see the numbers of training and testing pixels for each spectral class.
- Slide 2.10.6** The next step is to design the classifier, including the choices of kernel and multiclass strategy, and the values of the kernel and regularisation parameters. The OAO multiclass strategy was used, which required 36 different binary SVMs.
- Slide 2.10.7** A grid search procedure is commonly used to find optimal values for the kernel and regularization parameters. Here a slightly simpler approach was employed which involved two linear searches. The same parameter values were used in all 36 classifiers.
- Slide 2.10.8** This slide shows the results as a thematic map and as a table of class-by-class accuracies. Overall the average accuracy was 76.9%. Although the water class was handled perfectly, the performance on the rock and bare soil classes was quite poor and would not be acceptable if those classes were of interest.
- Slide 2.10.9** If we compare the thematic map to the image, we can see the classification errors in the rock and bare soil classes. Some of the rock pixels have been labelled as "tree" whereas many of the bare soil classes have been labelled as asphalt. This is a very crude accuracy assessment. We will be much more precise when talking about classifier errors when we come to module 3.

- Slide 2.10.10** We now look at the results of a classification of hyperspectral imagery, a problem for which the SVM is said to be more suitable than most other classifiers. It is taken from a 2004 paper and involves a 220 band data set called Indian Pines, recorded by the AVIRIS sensor
- Slide 2.10.11** This slide shows the image itself and a ground truth map—i.e. a map of pixels which have been labelled by site visits, use of photointerpretation and other sources of reference data. Only 9 of the 16 classes were used in this exercise
- Slide 2.10.12** The OAA multiclass strategy was used, as was the radial basis function kernel. By running a number of trial classifications, optimal values of the regularisation and kernel parameters were determined. The numbers of training and testing pixels are also shown here.
- Slide 2.10.13** Although we don't have a thematic map in this case, we do have a table of accuracies, showing remarkably good results for all classes. Note the interesting sensitivity analysis, which, at least for this example, indicates that the performance is not strongly dependent on getting the values of the kernel and regularization parameters exactly right.
- Slide 2.10.14** Remember the first point. The classes we are interested in may not always be those most easily resolved by a classifier. In a real remote sensing exercise, you may need to spend a little time ensuring that there is a link between the cover type labels in which you are interested and the classes most easily handled by a classifier.
- Slide 2.10.15** These questions lead you to thinking about how powerful the various classification techniques are, and a practical matter to do with parameter searching.

Lecture 11. The neural network as a classifier

- Slide 2.11.1** We now come to the fourth of the classification techniques we are going to consider in this course. We will develop it in two stages—in the original form and its return in a more powerful and flexible form over the past 5-10 years.
- Slide 2.11.2** The neural network, sometimes called the artificial neural network (ANN), was popular in remote sensing in the 1990s but because it was complex to design and required significant computational power it was overtaken by other techniques such as the SVM. However, with some simple modifications that lead to improvements in performance and training, it has gained popularity again over the past decade. Now called convolutional neural networks, these later variants also go under the name of deep learning, although that description could just as easily have applied to the NN in its original form.
- Slide 2.11.3** As with the SVM we start by a return to the simple linear classifier. Again, we will do our development with a simple two-dimensional two class situation, but it will generalize to any number of classes and dimensions. And again, we use our standard decision rule for linear decisions, which we will now represent diagrammatically in the form of what is called a TLU.
- Slide 2.11.4** In this diagram the elements up to the thresholding block create the linear function used in the decision rule. The thresholding operation then checks whether the value of the function is positive or negative, and thus whether the pixel vector is in class 1 or class2, as required by the algebraic expression of the decision rule. Sometimes, we represent the overall operation by the single block called a TLU. As noted on the previous slide this was one of the building blocks used in early machine learning theory that led to a classification machine called the Perceptron. It is also where we start the development of the neural network.
- Slide 2.11.5** A breakthrough came when the hard limiting operation in the TLU was replaced by a softer function and, in particular, one that could be differentiated. As we will see that allows a training procedure to be derived, which is not otherwise possible. We call the soft limiting operation an activation function; typical examples include the inverse exponential operation and the hyperbolic tangent as shown in this slide. As seen, the behavior still represents what we want in terms of specifying the class in which a pixel belongs because it implements our decision rule, but without a hard limit.
- Slide 2.11.6** The old TLU, but with a soft limiting operation, is now called a processing element (PE). In the nomenclature of neural networks, we replace the offset w_{N+1} by the symbol θ .
- Note in the bottom right hand drawing that we write the output of the PE as $g = f(z)$ where f is the chosen form of the activation function and z is the linear function in our normal decision rule.

Slide 2.11.7 The classical neural network, and that which was widely applied in remote sensing, is called the multi-layer Perceptron (MLP); it is composed of layers of PEs which are fully connected with each other, as seen in this slide. The blocks in the first layer are not actually PEs; they just distribute the input pixel vector elements to each of the PEs in the next layer

The outputs of those PEs then form the inputs to another layer of PEs, and so on, for as many layers as chosen by the user. The outputs from the last layer of PEs determine the class of the pixel vector fed into the first layer. The user can choose how that is done. Options are that each class could be represented by a single output, or the set of outputs could represent a code that specifies the class.

Note the nomenclature used with the layers of a neural network. In particular, the first layer which does any analysis is the (first) hidden layer.

Note also the letter designations we apply to the layers.

While we have shown only one, there can be many hidden layers, each being fed from the outputs of the previous layer. As the number of hidden layers increases, training the NN becomes increasingly time consuming. In many remote sensing exercises of several decades ago, only one hidden layer was used and found sufficient to handle many situations. But training still took a long time, as we will see later.

Slide 2.11.8 Having chosen a topology, we now need to work out how to find its unknown parameters (the weights w and offsets θ for each processing element). As with all supervised classifiers that is done through training on labelled reference data, but to make that possible we need to understand the network equations, so that a training process can be derived. That is now our immediate objective.

Slide 2.11.9 The equations describing each processing element are those we noted earlier—

$$g = f(\mathbf{w}^T \mathbf{x} + \theta).$$

However, we need a naming convention to keep track of where the inputs come from and where the outputs go. We do that by adding subscripts to the weights and offset as shown here. Our nomenclature is simplified in that it is layer specific but not PE specific. We could add a third subscript to indicate each actual PE in a layer but that turns out to be unnecessary—we can derive the relevant network equations for training without going to that added complexity.

Slide 2.11.10 To start developing our training procedure we need a measure of what we are trying to achieve. Clearly, if the network is functioning well as a classifier, we want the output to be accurate when the network is presented with a previously unseen pixel vector. We check the network's performance by setting up an error measure—a measure that looks at how closely the actual output matches what we expect when a training pixel is fed into the network.

We choose for that measure the squared-difference error measure shown in the centre of this slide. Remember the set of actual outputs of the network are the g_k . This measure tells us how well those actual outputs match the desired or target outputs for given training pixel vectors.

In the next lecture we will use that expression to help set up the necessary equations with which we can train the neural network. Clearly, our objective is to find the unknowns (weights and offsets) that minimize the error measure—in other words, that make the actual class labels match as nearly as possible the correct (or target) labels.

Slide 2.11.11 In this course we are examining the use of the popular MLP as a classifier in remote sensing. It consists of layers of PEs, where each PE contains a differentiable activation function. We are now at the stage where we can set up the network equations.

Slide 2.11.12 The second question here is important because it starts to develop a feel for the complexity of training a neural network.

Lecture 12. Training the neural network

Slide 2.12.1 We now look at how we can train a neural network. While this analysis is important in its own right, it turns out that the same process can be used when we come to look at the more modern convolutional neural networks.

Slide 2.12.2 We now wish to devise a training process for the neural network by seeking to minimize the error function we set up in the last lecture.

We do that by making small adjustments to the set of weights, such that those adjustments lead to a reduction in the error. The approach commonly taken is to modify a weight by subtracting a small amount of the first derivative of the error from its initial value, as shown in this slide. The idea is that by doing so we will move down the error curve, as indicated. This is called the gradient descent approach; there are other adjustment procedures, but the simple gradient descent method is good for illustration.

The amount of adjustment is controlled by the parameter η which is called the **learning rate**. A large value of η leads to greater adjustments but may lead to instability (oscillating between both side of the error curve in the illustration), whereas a too small a value means training time in lengthened.

The adjustment here is shown for the weights that link the j and k layers.

Slide 2.12.3 To work out a value for the adjustment to the weight component, we need to perform the differentiation. That is done by the chain rule, as shown in the centre of this slide. We now have to get vales for each of the derivatives in the chain rule expression.

Slide 2.12.4 Here we show each of those three derivatives and the final result when combined in the chain rule. Choosing $b=1$ in the activation function we end up with the correction increment shown on the bottom of this slide. We call that equation (A) for later convenience.

Slide 2.12.5 We now move to the front of the network and look to find the correction increments for the weights which link the i and j layers. We use the same gradient descent procedure as before to do that.

Here we have a small problem. Since E is not directly a function of g_j we cannot compute the derivative $\frac{\partial E}{\partial g_j}$ simply. Instead, we need to use another chain rule expression as shown on the bottom of this slide.

Slide 2.12.6 To get a value for $\frac{\partial E}{\partial z_k}$ in this expression we again use the chain rule which, with $b=1$ again gives an expression for $\frac{\partial E}{\partial z_k}$ in terms of the actual and target outputs.

That leads to the expression for the correction increment for the i to j weights as shown, which is not only a function of the actual and target outputs but is also dependent on the k to j linking weights. From the previous step we know those so that we now have a usable expression for the Δw_{ji} correction increments.

With the two sets of analyses we can now formulate a training algorithm of the neural network.

Slide 2.12.7 We can simplify our two previous equations if we define some new variables δ_k and δ_j which allow the correction increments for the two sets of linkages to be written simply as shown at the bottom of this slide.

While our equations are specifically focused on adjustments to the weights, the thresholds θ_j and θ_k in the network equations can be evaluated using the same expressions, just by making the corresponding inputs unity during training.

Slide 2.12.8 We now formulate the training strategy.

The chosen network is initiated with an arbitrary set of weights. That allows outputs (although in error) to be generated by the presentation of training pixel vectors at the input layer.

For each training pixel the network output is computed from the set of network equations. Initially, the output will be in error.

Correction to the weights is then performed using the equations of the previous slide.

The value of δ_k is computed first since it depends on the network outputs g_k compared with the target outputs t_k . Remember $\delta_k = (t_k - g_k)(1 - g_k)g_k$.

Then the result can be propagated back through the network, layer by layer (if there is more than one hidden layer) using the other equations on the previous slide to generate corrections to the network weights. Specifically, $\Delta w_{kj} = \eta \delta_k g_j$ can then be found, following which we can get δ_j and then Δw_{ji} .

Slide 2.12.9 When all the weights have been adjusted the output of the network is computed again using the new weights. Hopefully the g_k will now be closer to the target values t_k . New values for the δ_k will then be generated, and the process of weight adjustment is repeated.

This process is iterated as often as needed to reduce the difference between the actual and target outputs ($t_k - g_k$) to zero, or to a value acceptably close to zero. If it is zero, then δ_k will be zero meaning that no further adjustments to the weights will occur with further iteration. Then network is then fully trained. In the terminology of neural networks, an iteration is called an **epoch**.

Because training involves working back from the outputs at each epoch (iteration) the training process is referred to as back propagation.

- Slide 2.12.10** The interesting thing about training the MLP is that when a training pixel is presented at the input, the calculations are propagated forward through the network to generate the output. That output is checked against the correct class for that training pixel and, if found to be in error, the equations we derived in this lecture are used to propagate backwards through the network the adjustments to the weights.
- Slide.2.12.11** The first question here draws your attention to the possibility of local minima in the error curve.

Lecture 13. Neural network examples

Slide 2.13.1 We will now look at some examples to illustrate the training and performance of the neural network

Slide 2.13.2 When considering the application of the neural network there are a number of decisions that need to be taken beforehand about the network topology. These include:

1. How many hidden layers to use
2. How many nodes should be used in each layer
3. How to use the output layer to represent the thematic classes
4. What value to assign to the learning parameter

Generally, the first layer will have as many nodes as there are elements to the pixel vector. Often, there will be as many output layer nodes as there are classes, unless some form of coding is used to reduce the number. Generally, the number of nodes in the hidden layer should be not less than the number of output layer nodes.

Note that the MLP has all the connections in place that we described earlier—i.e. the output of a PE (node) in any layer is connected to every node in the next layer. That is called fully connected. Later, in the context of the convolutional neural network, we will not use all of those connections.

Slide 2.13.3 First, we start with a very simple example, involving two classes in a two-dimensional vector space. Note that the classes are not linearly separable.

We have chosen a network with two nodes in the hidden layer and two in the output layer. The network equations are shown explicitly on the right-hand network diagram. Note also that we have chosen a zero threshold θ for the hidden layer PEs, $b=1$ in the activation function and $\eta=2$.

Slide 2.13.4 The network was initialized with the set of weights shown in the first row of this table. As seen, the error before iteration was 0.461.

The network was then trained for 250 iterations, at which the error had been reduced to 0.005. At the same time the weights can be seen to be converging to fixed (final) values.

Slide 2.13.5 We stop training at 250 iterations and use the parameter values at that point. On the right hand axes we have plotted the arguments of the two hidden layer PEs before application of the activation functions—effectively when equated to zero they implement linear separating surfaces. The activation function then places the response for a given pattern on either side of those surfaces.

Each surface segments the data space into two regions. The output layer PE takes those responses to segment the space into the two class regions. Effectively, it implements a logical OR operation. That is shown mathematically in the bottom table which shows explicitly how the output layer functions for each of the four possibilities of patterns being placed either side of the first two surfaces.

- Slide 2.13.6** Having trained the network we need to see how successful it is in separating classes of pixel vector that it has not seen. In the table there are 8 new patterns. They can also be seen in the vector space. Patterns a,b,c,d are in class 1 while e,f,g,h are in class 2, as is evident on the diagram. The table shows the intervening calculations and the final classification by the network for each pixel. All testing pixels have been successfully labelled.
- Slide 2.13.7** We now come to a real remote sensing example, taken from the 1995 paper listed on this slide. The data set consisted of the six non-thermal bands of a 900x900 pixel segment of the thematic mapper scene recorded over Tucson, Arizona on 1 April 1987. There are 12 evident classes in the scene that were chosen by the authors. The band 4 (near IR) image shown here does not make those classes easily seen but the grid structure of Tucson's streets is evident.
- Slide 2.13.8** In keeping with most remote sensing exercises of the time involving neural networks, the authors chose a network with one hidden layer. Since there were 6 bands, the input layer consisted of six nodes, which also scaled the data to the range (0,1).
 Since there were 12 classes the output layer was chosen to have 12 nodes, with each representing a single class. The scale of the outputs was chosen such that during training an output of 0.9 on a node indicates a target class while a value of 0.1 means that that class does not correspond to the training pixel being presented. The hidden layer was chosen to have 18 nodes. Since they decided to compare the neural network results against those obtained with a maximum likelihood classification their choice of the hidden layer nodes was based on having the same number of parameters to determine as for the maximum likelihood classifier.
- Slide 2.13.9** This slide shows the information classes and the numbers of training and testing pixels used by the authors.
- Slide 2.13.10** Although the network was allowed to run for 50,000 iterations (or epochs) the error had stabilized after about 15,000 iterations. Note that more than 96% of the training pixels are properly handled once the network has reached that number of iterations.
 It is because so many iterations are needed to train a neural network that training time can be excessive.
- Slide 2.13.11** The network performance using unseen testing pixels was a very good 93.4% accuracy. If training was stopped after 10,000 iterations the network was still capable of achieving 92% accuracy; if stopped at 20,000 iterations that improved marginally to 93%.
 A maximum likelihood classifier was run on the same data set, although there is no indication as to whether it was optimized through the choice of sets of spectral classes to represent the specified information classes (which we will do in module 3). Nevertheless, the maximum likelihood classifier achieved 89.5% accuracy on the testing data, but it was 10 times faster to train.

- Slide 2.13.12** This slide shows the thematic map produced by the neural network on the right-hand side, along with the key to the colours.
- Slide 2.13.13** The authors included two variations to the standard neural network training process to improve the learning phase. The first was to add a “momentum” term to the gradient descent rule used to adjust the weights. On the top of this slide we summarise the standard gradient descent adjustment. On the bottom (in green) an additional term is added. It is chosen as a proportion of the previous weight adjustment, which forces the modification to follow the pattern of the previous iteration. Another parameter is introduced in this process— α which controls the degree of momentum used.
- Slide 2.13.14** The second modification was to adjust the learning and momentum rates adaptively in order to improve convergence. That was done every fourth iteration according to the rule shown on the top of this slide.
- Note that the convergence and ultimate result of neural network training can be affected by the initial choice of weights, and that the initial set cannot all be the same, otherwise the network will not train.
- Slide 2.13.15** More details on this example will be found in the paper. However, this original neural network approach is now rarely used. We introduced it here as preparation for the more recent development of the convolutional neural network, which we commence in the next lecture.
- Slide 2.13.16** When we come to the convolutional neural network we will often talk about deep learning. Simply put, network depth is described by the number of hidden layers. A deeper network has more. The idea is that when there are more hidden layers the network should be more powerful. The network is then more difficult (time consuming to train) because of the vastly larger numbers of unknowns that have to be found. When we come to the convolutional neural network, we will find that increased network depth is possible because we don't use all the connections between the nodes. By reducing the number of connections substantially we can have more layers and still train the network, even though that can still be time consuming.
- Slide 2.13.17** As a final comment on the operation of the layers in the neural network, this slide gives a different perspective on how the simple network of the first, simpler example operates. Earlier, we regarded the hidden layer PEs as implementing two linear decisions, with the third layer acting on those decisions as a logical OR function. We can also view the hidden layer operation as in this slide, if we examine the data as it appears at the output of the first layer PEs. Now represented by the variables g_1 and g_2 the data has been transformed into a linearly separable set, which the output layer now handles.

- Slide 2.13.18** Here we summarise the transformation properties of the neural network and some design guidance.
- Slide 2.13.19** The second and third questions here will become important when we look at the convolutional neural network in the next series of lectures. What properties does it have to have in order that the back propagation training algorithm can be made to work?

Lecture 14. Deep learning and the convolutional neural network, part 1

Slide 2.14.1 We now start a series of four lectures on the transition of the neural network that we met in the past few lectures into the convolutional neural network that has become a cornerstone of artificial intelligence research over the last few years. It has also been widely applied to remote sensing problems, as we will see when we look at some examples. At the completion of this work, we will then do a detailed comparison of the major classification techniques we have covered in the course.

Slide 2.14.2 As we will see, by comparison to the classifiers we have looked at so far, the CNN does not have a standard form (or topology). Instead, it is composed of a number of building blocks that can be configured in many ways according to the approach chosen by a particular user. We will develop the tools and show several configurations helpful in remote sensing, especially for taking the spatial neighbourhoods of pixels into account when performing thematic mapping.

The book by Goodfellow and the others referenced here has become a bit of a standard treatment for deep learning and CNNs. Its treatment is at a higher mathematical level than we have adopted in these lectures, but nevertheless, for those with the right background, it should be consulted to fill in the gaps in theory. Note especially its warning, listed here, about non-standardization in CNNs!

Slide 2.14.3 Before we embark upon the development of the CNN it is important to reflect on the fact that, for many decades, image analysts in remote sensing have been critically aware of the matter of spatial context. That is: when considering the label for the central pixel in the diagram on the slide, we know, for many scenes, that there is a high likelihood that the surrounding pixels will be from the same class. That is especially the case for agricultural regions and many natural landscapes.

And yet the classifiers we have been dealing with up to now have ignored that property. In that sense they are called **point** (or pixel-specific) classifiers, because they just focus on a pixel, independently of its neighbours.

Slide 2.14.4 Over the years, though, there have been many suggestions about how to treat spatial context. Some of the more successful approaches are shown here. EHCO was perhaps the earliest. Developed in the mid-1970s it works by growing homogeneous regions in an image. It then classifies those objects (or regions) as a whole. It applies point classification methods for pixels that are found not to be part of an object, such as on the boundaries between regions.

In the late 1970s the method of relaxation labelling was developed. It takes the results of a point classification, but expressed as posterior probabilities of class membership, such as in the output of a maximum likelihood classifier. Those posteriors are updated iteratively by reference to the posteriors on the neighbours, linked via a set of joint probabilities.

Finally, measures of **texture** can be used to characterize the neighbourhood about a pixel. Local texture is then used as another feature in a point classification method, along with the spectral measurements of a pixel.

As we will see soon, the convolutional neural network is another technique that embeds spatial context in its decisions.

Slide 2.14.5 To employ the neural network in *spatial context sensitive applications* we have to use it in a slightly different way than we have up to now. Let's commence this discussion by recalling the topology we have been dealing with so far, in which the inputs are the individual components of the pixel vector.

Slide 2.14.6 Suppose now we make the seemingly bold move of inputting **all** the pixels of an image in one go, so that we have to have enough input nodes to accommodate the *full set* of spectral measurements for the *full set* of image pixels. For a practical image that will be a very large number of inputs. We still have a number of hidden layers and, for the moment, the network is fully connected. Thus, there will be a huge number of unknown weight vectors and offsets to be learned through training.

One immediately obvious problem with feeding the network in this manner is that the spatial inter-relationships among the pixels appears to be lost. Even though this is really just a problem of how the pixels are addressed, it is more meaningful to arrange them as shown in the next slide.

Slide 2.14.7 Suppose we present the image to the network as a square (or rectangular) array, with the pixels in their correct spatial relationships. This doesn't change anything about the network, other than arranging the nodes (or processing elements) into an array rather than in column format. For convenience we have shown the hidden layers to be the same size and shape as the input layer, but in general they could be any size. Note the output layer is still one dimensional, since it represents a set of classes. We also assume for the time being that we are dealing with a single band image, so each input node is a single scalar value.

Slide 2.14.8 With such an arrangement the number of potential connections is enormous. Let's do a calculation of the number of unknowns between just the input and the first hidden layer. Remember that the input to each processing element in the *hidden* layer is $z = \mathbf{w}^T \mathbf{x} + \theta$. The dimensionality of the weight vector will be equal to the number of elements in the input layer, which is $N \times N$. Also, there are as many weight vectors as there are nodes in the hidden layer. If we assume, for the sake of this calculation, that the hidden layer has the same dimensions as the input layer, that means altogether we have N^4 different weights, values for which have to be found during training to make the network usable. In a similar fashion there will be N^2 values of θ .

If we had $N=100$, which would be a very small image in remote sensing, then there are more than 100 million unknowns. That would require an extraordinarily large amount of training data. Added to this is the fact that we have multiple bands, and images usually much larger than 100×100 .

Clearly, a simpler approach needs to be found, but one in which spatial inter-relationships among the pixels are still represented.

Slide 2.14.9 In this slide we just simplify the diagram by removing the explicit input layer and just let it be represented by the image itself, perhaps with scaling if that is found to be beneficial in some applications. In this simplified representation each image pixel connected to all the nodes of the first hidden layer.

Also, we are still **focusing on just a single band of data**, so that each input node is just a single number. We will come to multispectral images later.

Slide 2.14.10 Here we show the major deviation of the convolutional neural network from the fully connected NN we have been considering so far. Instead of implementing all connections—i.e. as in a fully connected network—we are selective in the connections we make between layers. In particular, we restrict the connections to a node in the hidden layer to be just those of a neighbourhood of nine pixels from the input image, as shown.

Because of the geometry, the group of 3x3 pixels is centred on the one which is in the second row and second column. The PE element is also that in the (2,2) position as seen in the slide.

In contrast to the need to determine $N^4 + N^2$ weights and offsets overall there are now 10 unknowns (9 weights w_{ij} and one offset θ) to determine per hidden layer node. Overall, therefore there are, in principle, $10N^2$ unknowns to find, a considerable reduction, but still a large number if N is large.

Slide 2.14.11 We do the same for the 3x3 group which is one column to the right. Now we take a decision *that significantly reduces again* the number of unknowns to be found in training: rather than use a new set of weights and offsets, we assume we can employ the same set as for the previous slide. This is called *weight re-use*, and while that sounds like it will reduce substantially the power of the network to learn complicated spatial patterns in the image, it gives surprisingly good results in practice. There is also a rationale to this decision which we will see soon.

Slide 2.14.12 Continuing though, we then do the same thing for the next pixel group along the row.

Slide 2.14.13 And then for all rows until the whole image is covered. While this example suggests that the actions happen sequentially, in fact all the operations are in parallel—they are just sets of connections. This is important to recognize.

As we have realised there is a problem with the edge pixels. Given the large numbers of pixels in an image we could ignore the edge problem. Sometimes an artificial border of zeros is created so that the edge PEs in the hidden layer can receive inputs and thus preserve dimensionality, if that is important.

Even though many of the connections of a fully connected NN have now been removed, it turns out *we can still use back propagation* to train this new, sparser network.

Slide 2.14.14 Let's summarise where we are at this stage. In looking at the CNN we are partly driven by the desire to take spatial context into account when labelling a pixel. In the CNN all the image is fed to the network in one go, but the numbers of node-to-node connections is greatly reduced, and thus so are the number of unknown parameters to be found during training.

Slide 2.14.15 The first question here asks you to think about the importance of spatial context. The last two questions are particularly important when thinking about the use of CNNs.

Lecture 15. Deep learning and the convolutional neural network, part 2

Slide 2.15.1 In this lecture we take the development of the CNN further, still focusing just on a single band of data, but considering the evolution of its topology.

Slide 2.15.2 The concept we adopted in the last lecture for the connections between layers is similar to the common process of convolution used to filter an image to detect spatial features. We haven't covered that material in this course, but it is moderately straightforward.

In spatial convolution, a window, called a kernel, is moved over an image row by row and column by column. A new brightness value is created for the pixel under the centre of the kernel by taking the products of pixel brightness values and the kernel entries, and then summing the result. That is exactly the same operation implemented by a processing element in the hidden layer of the CNN just before the offset is added and the activation function is applied.

It is because of that similarity that the partially connected neural network just described is *called* a convolutional neural network (CNN).

However, in the CNN the kernel is usually called a *filter*, and the set of input pixels covered by the filter is called a *local receptive field*. Note that any size filter and receptive field can be used.

Slide 2.15.3 Even though we are exploring the smaller number of connections as a way of simplifying the network, *and thus* the number of unknowns that need to be found during training, it is of interest to think a bit further about the practical significance of choosing a (spatial neighbourhood) kernel of weights for that purpose. While important in our analysis of spatial context, this has particular relevance to picture processing and object recognition, fields in which the CNN have been used extensively over the past five years or so.

In spatial filtering, say for detecting the edges in an image, the kernel, or filter, entries are selected by the analyst for that purpose, as seen in this very simple example. A 3x3 filter can be used to find the edges in an image.

Slide 2.15.4 In the CNN the kernel entries (i.e. the weights prior to the application of the activation function) are initially chosen randomly. However, by training, they take on values that match the image features that are characterized by the spatial nature of the training samples. If the training images strongly feature edges, it is expected that the weights will tend towards those of an edge detecting filter, for example. The strength of the CNN is that with a sufficient number of layers it can learn the spatial characteristics of an image. That is why it is an important tool for performing context classification and for picture processing in general.

Slide 2.15.5 We now introduce some more operations used in CNNs, along with their associated nomenclature. The first is the concept of *stride*. When we looked at feeding just nine outputs from one layer into a single PE of the next layer, we did so with single pixel shifts along rows and down columns. Some authors choose to have larger shifts, the result of which is that the number of nodes in the next layer is reduced. The number of pixel-shifts is what defines **stride**. This slide shows a stride of 2.

Slide 2.15.6 Another topological element often used is to add so-called **pooling layers** as seen on the right-hand side of this slide. This strengthens the dependence on neighbourhood spatial information and reduces further the number of parameters to be found through training, particularly when more than a single convolutional (hidden) layer is used. Pooling is sometimes called down-sampling.

Slide 2.15.7 We then have a decision as to how to proceed further and ultimately construct an output for the CNN. There seem to be four common options:

1. Keep going by feeding into another convolutional layer, to provide a deeper network; we can, in principle, have as many layers as we wish: just like with the fully connected network we can have as many hidden layers as we like.
2. Feed the hidden layer output values into a set of output layer PEs and thus terminate the network
3. Use the outputs as the inputs to a normal fully-connected NN, in which case the CNN acts as a feature selector for the NN; this is a common approach, especially in remote sensing
4. Generate a set of class probabilities from the outputs.

Let's examine the last two.

Slide 2.15.8 Here we show a network with two convolutional layers and one pooling layer feeding into a (much smaller) fully connected NN of the type we considered a couple of lectures ago. In effect, the CNN is acting as a feature selector for the fully connected network. Note that we have introduced another term “flattening”—that is just the process of straightening out the matrix into a vector, as needed for the NN input.

Slide 2.15.9 After flattening, rather than feeding the results into a fully connected network another, very common option is to use the CNN outputs to generate a set of pseudo-probabilities, called softmax probabilities, defined as shown. The CNN outputs are exponentiated and normalized as shown, so that the set of softmax values replicate a set of posterior probabilities.

Slide 2.15.10 Finally, the sigmoid activation function is usually replaced by a simpler activation function called the ReLU—the Rectified Linear Unit—which has the characteristic shown. This speeds up training by improving the efficiency of the gradient descent operation used in back propagation.

Slide 2.15.11 Note that the use of stride and pooling successively reduces the number of unknowns to be found by training. Also, note that convolutional and pooling layers can be cascaded.

Slide 2.15.12 The second question here leads to one of the design equations used with CNNs.

Lecture 16. Deep learning and the convolutional neural network, part 3

- Slide 2.16.1** In this lecture we confront the problem of multi-dimensional images—colour pictures made up of the three colour primaries, and multispectral and hyperspectral images in remote sensing.
- Slide 2.16.2** In this slide we see the three colour primaries of a colour picture. Alternatively, they could be three bands of a multispectral image. We describe the image pixels as shown by the three equations on the top of the slide. On the bottom we show the corresponding three filter entries. In both cases we have given three indexes. The first refers to the individual band, while the others are the pixel index.
- Slide 2.16.3** The simplest way to treat the three band Image is to carry out three separate convolutions as shown by the equations on the top of this slide. Generally, only a single offset θ is used. The three convolution calculations are added to which the offset is also added, and then the activation function is applied. We now have three times the number of weights to learn via training.
- While this is the approach most often adopted for colour pictures we will see, later, how multispectral and hyperspectral images are treated.
- Slide 2.16.4** Here we show another variation often used in the convolutional neural network. Several convolutions can be performed in parallel in order to extract more spatial information from an image. As noted, the filters can be of the same or different sizes.
- Slide 2.16.5** Because of the complexity introduced by the various options we have discussed, it is difficult to come up with a standard form of diagram with which to represent the convolutional neural network.
- Many authors use their own forms of diagram. The representation shown here is common to many and simple to understand.
- Here we show convolutions in parallel, as just discussed on the previous slide. We also show several layers, each of which is composed of a convolution operation follow by pooling. Of course, the pooling operations are not essential but are included here for completeness.
- Finally, we show the flattening operation often used at the output. As indicated, some authors even have crossed connections between the parallel paths. That can defeat one of the benefits of the convolution neural network: by having several separate parallel paths the network can be programed to run on multiple processor machines simultaneously.

Slide 2.16.6 We now come to an important practical consideration, similar to that we met with the maximum likelihood classifier when considering the Hughes phenomenon. And that is the problem of over fitting, is illustrated on this slide.

The concern arises because we have so many weights and offsets to be found through training, and the availability of training data determines how effectively those unknowns can be found. We must have sufficient training samples available to get reliable estimates of the unknown parameters, otherwise the network will not generalize well. In other words, it will not perform well on previously unseen pixels.

It is not sufficient to have a minimum of samples to estimate the unknowns, otherwise over-fitting will occur. This is illustrated in the example from curve fitting shown in the diagrams. Fitting a high order curve through just three points, will guarantee good fits for those points, but the behaviour between the points can be *way out* in terms of being able to represent intervening points not used in generating the curve. If many “training” samples are used then the function found interpolates (i.e. generalizes) well.

Clearly, we need many more training pixels than the minimum to ensure we do not strike the same problem when training the neural network.

Slide 2.16.7 Consider now the numerical complexity of analyzing hyperspectral image data, in which we want make use of both spectral and spatial properties.

Several approaches have been used in practice, as we will see shortly in some examples. One is to analyse the spectral information content alone. Another is to analyse the spatial information content alone (i.e. spatial context). Another is to do both together. But there is a processing challenge.

Slide 2.16.8 We could treat the problem of processing hyperspectral data with a CNN by allocating one convolution filter to each band, as we did previously for the three band colour picture. But that requires about 200 times as many weights as for a single band image. For an image with 200 bands, and 3x3 kernels, the total number of unknowns (i.e. weights plus offsets) connecting the input image to the first convolutional layer is 2000, noting that the same weights are used in each filter right across a particular band. This, of course, gets multiplied upwards by the number of filters used in the convolutional layer.

Slide 2.16.9 Often, we take the option of reducing the spectral dimensionality of the hyperspectral image before applying the CNN. Although that partly defeats the purpose of using hyperspectral imagery, transforms such as PCA do allow us to concentrate the variance (or information content) in a small number of components. Three are shown here, but more might be necessary if we wished to retain say at least 95% of the image variance.

Slide 2.16.10 If we want to analyse hyperspectral data for spectral properties alone we can use the CNN to find a label for each pixel based just upon its spectrum, and thus implicitly the correlations between bands.

- Slide 2.16.11** Here we summarise how multiband images can be handled, right through to data as complex as hyperspectral imagery. Importantly, the need to avoid overfitting must be kept in mind at all times.
- Slide 2.16.12** The first question asks you to propose a simple formula, based on the discussion in this lecture on using PCAs.

Lecture 17. CNN examples in remote sensing

- Slide 2.17.1** We now present two examples that illustrate much of what we have discussed in these lectures and which help us introduce some additional concepts.
- Slide 2.17.2** The examples illustrate how CNN have been used to handle hyperspectral data, based on spectral properties alone, spatial properties alone and a combination of spectral and spatial properties.
- Slide 2.17.3** Our first example is taken from the paper indicated. It presents examples of hyperspectral classification on several data sets, **based on just the spectral properties** of a pixel. Here we look at a classification of the Indian Pines data set, which we saw before with the support vector classifier example. The data was recorded by the AVIRIS hyperspectral sensor over a region in Indiana, USA. It consists of 220 spectral channels in the range 0.4 to $2.45\mu\text{m}$.
- Slide 2.17.4** In treating this image, the authors chose to remove some difficult classes. They retained the 8 classes shown in the table which also indicates the numbers of training and testing pixels used.
- Slide 2.17.5** The authors chose to use a single layer CNN as a feature selector prior to classification by a fully connected NN. They applied 20 spectral filters in parallel and used a fully connected network with a hidden layer of 100 nodes. Note how large their filters are. Altogether there are 81,408 unknowns to be found from the training data. From the previous slide there were 1600 training pixels. At 220 channels per band that gives 352,000 training samples, which is sufficient.
- Slide 2.17.6** This slide shows their results in the form of a thematic map, and the accuracy achieved compared with that the generated with a support vector classifier. The accompanying ground truth map (i.e. a map of correct labels) allows one to assess how good the final thematic map is. Of note, though, is the speckled appearance of some classes, indicating that the CNN misclassified a number of pixels. Had it incorporated spatial filtering too, we would expect to see a much cleaner thematic map.
- Slide 2.17.7** The second example we will consider uses a two-channel CNN to account for both the spectral and spatial properties of hyperspectral scenes. One channel is devoted to spectral properties alone and functions very much as in the previous example. The other channel handles the spatial analysis. Both channels develop feature subsets that are concatenated and then analysed by a fully connected NN. The example is taken from the paper indicated.

- Slide 2.17.8** We are going to concentrate on their Salinas California image exercise. The image segment consists of 512x217 pixels, with 3.7m spatial resolution. It has 224 recorded bands but the authors reduced those to 200 by removing channels with poor quality. The ground truth image shows that there are 16 classes, with the numbers of pixels indicated.
- Slide 2.17.9** The authors chose to train the network using different percentages of the ground truth pixels. We show the results here for the training data being 25% of the total labelled ground truth pixels. They used all the available labelled ground truth pixels to test the generalisation of the network—i.e. the classification performance.
- Slide 2.17.10** This is the CNN topology, or architecture, used by the authors. It consists of a spectral path at the top and a spatial path at the bottom. Notice that the spatial path has 30 filters of size 3x3, and the spectral path has 20 filters of size 20x1. Each pathway has one convolutional layer and one pooling layer.
- The input to the spatial path consists of a spatial neighbourhood about the pixel currently under consideration during training, or in classification.
- The outputs from the two paths are flattened, concatenated and then fed into a fully connected NN with two hidden layers, each with 400 nodes. Thus, the two path CNN is acting as a feature selector to the NN.
- Note that the output layer has 16 nodes, representing the 16 classes in the Salinas image. The outputs are in the form of class conditional probabilities computed with the softmax function.
- Slide 2.17.11** There are two important aspects of this example which need to be emphasized.
- The spatial layer is required to capture the neighbourhood (or spatial) properties of a pixel. A neighbourhood patch of 21x21 pixels, centered on the pixel of interest, is used. The neighbourhood patch is created by averaging over all the spectral channels in that neighbourhood.
- The authors also used transfer learning. This is a technique based on the concept that networks previously trained on different images, but with the same sensor, will most likely perform acceptably on the image of interest. This is based on the assumption that the spatial properties are similar from image to image. The authors trained the CNN layers on a different AVIRIS image, and then used the weights so found to initialize the CNN weights for training on the Salinas scene. This is not necessary in general, but it is a common approach, based on the concept that we, as humans, adapt our learning from past experience.
- Slide 2.17.12** The results, shown here, indicate the benefit of using both spectral and spatial context, achieving an overall classification accuracy of 98.3%, which is very good!
- It is important to note that the authors ran extensive trials to find the best topology for the network—the numbers of convolutions layers, the numbers of filters, the numbers of nodes in the hidden layers, and so on, which indicates that the preparatory stages in using a CNN can be quite extensive.

- Slide 2.17.13** The idea of using neighbourhood patches seems first to have been introduced by Makantasis in the paper referenced in this slide. Those authors used 5x5 patches, but maintained the full spectral dimension for the patches, so that spectral information as well as the spatial neighbourhood of a pixel was carried by the patch. However, in order to constrain the overall data volume of the input, the authors carried out a dimensionality reduction first using a principal components transform.
- Slide 2.17.14** This slide gives information on where convolutional neural network software can be found.
- Slide 2.17.15** Two important points are emphasised in this summary. First, patches (neighbourhoods) can be fed to a CNN to carry spatial context into a classification. Secondly, transfer learning can be an effective and efficient way to initialise a CNN (and even a fully connected NN).
- Slide 2.17.16** These questions ask you to think carefully about some of the quantitative aspects of using a CNN.

Lecture 18. Comparing the classifiers

Slide 2.18.1 Having completed our examination of the most popular classifiers used in remote sensing, it is now of benefit to compare them, both in terms of performance and in relation to the user effort required.

Slide 2.18.2 We want to compare their attributes so that we know where their relative strengths and weaknesses lie, and so that we always choose the most appropriate method for the task at hand. Of the range of algorithms we have looked at, the following three are a representative set for comparison purposes, and are the ones we have spent most time on:

- The maximum likelihood classifier (MLC)
- The support vector machine (SVM)
- Convolution neural networks (CNN)

Slide 2.18.3 Let's commence by summarizing the maximum likelihood algorithm. Recall that the decision rule for allocating a pixel to a class is expressed in terms of discriminant functions.

Each class is defined by its mean vector and covariance matrix, and if available the class prior probability, and is represented by those properties in the discriminant function

Slide 2.18.4 The attributes of the MLC are summarized here.

Slide 2.18.5 The support vector classifier finds the separating hyperplane that maximises the margin between two classes while minimizing the error caused by pixels that fall on the wrong side of the hyperplane. It uses kernels in place of dot products effectively to project the data into a higher order space, so that data which is not linearly separable can be handled.

Slide 2.18.6 The attributes of the SVM are summarized here, noting particularly that it has to be used in a decision tree to make it capable of handling multiple classes.

Slide 2.18.7 The CNN is a modern variant of the original multilayer Perceptron. It consists of a number of layers, each usually involving sets of filters that perform convolution, activation (ReLU) and pooling. Those layers can then be followed by a fully connected NN and/or a softmax operation.

Slide 2.18.8 The attributes of the CNN are summarized here.

Slide 2.18.9 In this table we bring the most important attributes together so that the three principal algorithms can be compared. In summary the MLC is much simpler to construct and train but is limited when presented with data of high spectral dimensionality.

By comparison the SVM and the CNN are more challenging to configure and train. The SVM is good for handling data of high dimensionality but requires a decision tree framework to handle more than two classes. The CNN naturally handles spatial context and, like the MLC, is a multi-class algorithm.

Slide 2.18.10 Here we summarise what the user needs to look for in selecting an algorithm for thematic mapping in remote sensing.

Slide 2.18.11 Again, these test questions draw attention to the types of application the analyst may have to handle, leading to a choice of the most appropriate classifier algorithm.

Lecture 19. Unsupervised classification and clustering

Slide 2.19.1 We now turn to the topic of unsupervised classification, in which we are still interested in thematic mapping but without the benefit of having labelled training data beforehand. We will develop the topic in this series of lectures, based on the procedure called clustering. Indeed, most of what we will talk about concerns clustering algorithms, but we will present some unsupervised clustering examples. We will meet unsupervised clustering again in module 3

Slide 2.19.2 We start by being confronted with the situation in which no obvious training data is available and yet we still want to do thematic mapping of remote sensing image data. Cluster analysis forms the backbone of what we are going to do.

Clustering looks for groups of similar pixels, assessed on the basis of their spectral properties. So, the groups we search for are clusters in spectral space.

We can often identify the pixel labels produced by clustering through the use of spatial clues in the image and by using the cluster means as surrogates for spectral reflectance information. We will see that in the examples to follow.

Reference data, like maps and air photos, also give us hints as to what the classes in a cluster map might represent.

Slide 2.19.3 Here we look at unsupervised classification as a two stage process, in which clustering takes us from the spectral domain to a map of symbolic labels. The challenge for the analyst is to turn those symbols into meaningful ground cover labels.

We get the cluster map in the same way we got a thematic map in supervised classification. Here the clustering algorithms place pixels into clusters of similar pixels (based on spectral similarity). We then assign symbols to the clusters and use those symbols in place of the pixel itself in the image, thus producing a cluster map. By examining those pixels, and their spatial layout, the analyst turns the symbols into ground cover class labels.

Slide 2.19.4 Clustering algorithms place pixels into clusters based on their similarity. As we indicated in the previous slide, the most common measure of similarity is based on the spectral measurements of the pixels. Two pixels with very similar measurement vectors are likely to belong to the same class and thus cluster.

The simplest way of assessing similarity is to use a metric which measures the spectral distance between pixels, the most common of which is the Euclidean distance between pixels in spectral space.

For N band data the Euclidean distance metric is

$$\begin{aligned} d(\mathbf{x}_1, \mathbf{x}_2) &= \|\mathbf{x}_1 - \mathbf{x}_2\| = \{(\mathbf{x}_1 - \mathbf{x}_2)^T(\mathbf{x}_1 - \mathbf{x}_2)\}^{\frac{1}{2}} \\ &= \{\sum_{n=1}^N (x_{1n} - x_{2n})^2\}^{1/2}. \end{aligned}$$

Slide 2.19.5 There are other distance metrics in use, including the more efficient but perhaps less accurate city block distance $d(\mathbf{x}_1, \mathbf{x}_2) = \sum_{n=1}^N |x_{1n} - x_{2n}|$, which just sums the absolute differences by band and is similar to walking between two locations in a city where the streets are laid out on a rectangular grid. It is sometimes called the Manhattan distance.

In these lectures we will concentrate on Euclidean distance.

Slide 2.19.6 The set of clusters for a given image is not unique, even though we have a metric for spectral similarity. Here we see two possible, plausible clusterings of eight pixel vectors in a two dimensional spectral space. Which one is correct?

Slide 2.19.7 To help assess which set of clusters best represents the pixels in an image, we need a “quality of clustering” criterion; a common one is the sum of squared error measure (SSE) with the formula shown here.

The SSE checks the distances of all the pixels in a given cluster from the cluster mean, and it then sums those distances within that cluster. It does so for all clusters and then sums the results. In other words, it is an accumulative measure of distances of the pixel vectors from the cluster means. The smaller it is the better, since then the clusters are compact.

Other quality of clustering measures are possible. Some look at the average compactness of clusters compared with the average distances among them.

Slide 2.19.8 In principle we should be able to develop a clustering algorithm by minimizing the SSE for a given data set. But that turns out to be impractical, since it would require examining an enormous number of candidate clusterings of the available data to find that with the smallest SSE. In practice, some heuristic methods have been developed that work well, two of which we will look at here.

The first is the k means or migrating means algorithm. It is perhaps the most commonly used, particularly for remote sensing problems.

It asks the user: (1) to specify beforehand how many clusters to search for, and (2) to specify a set of initial cluster mean vectors. Clusters are located in spectral space by their mean vectors; the algorithm starts by the user guessing a set of cluster centres.

The image pixels are then assigned to the cluster of the closet mean, after which the set of means is re-computed. The pixels are then assigned to the nearest of the new set of means, and so on until the means and assignments do not change.

- Slide 2.19.9** This slide shows algorithmically the steps in the k means approach.
- In the first two steps clustering is initiated by specifying a starting set of cluster mean vectors, both in number and position.
- In step 3, all the available pixel vectors are then assigned to the cluster of the nearest mean; the mean vectors are then re-computed in step 4. Then a new assignment of pixel vectors is carried out based on the re-computed means. The means are then re-computed again. During this process pixels will often move between clusters, iteration by iteration, because of the changing positions of the means.
- Ultimately, we expect that the stage will be reached where the pixels do not migrate any further between the clusters, so that the situation is stable and we conclude that the correct set of clusters has been identified. That is checked by seeing whether the full set of mean vectors no longer changes between iterations.
- In practice, we may not be able to wait until full convergence has been achieved, and instead we stop clustering when a given (high) percentage of the pixel vectors no longer shifts between clustering centres with further iteration.
- Slide 2.19.10** Before we implement the k means technique in the next lecture we here take stock of what we are trying to achieve. Our ultimate goal in many remote sensing situations is unsupervised classification, which we pursue through the application of clustering techniques.
- Slide 2.19.11** Even though we haven't yet seen an example of the application of the k means approach, we can still think about some practical matters concerning its operation, and especially how we choose the initial cluster centres. That choice will affect the speed of convergence of the algorithm and, indeed, the actual cluster set found.

Lecture 20. Examples of k means clustering

Slide 2.20.1 In this lecture we look at two examples of k means clustering. The first uses a small data set to show how the algorithm operates. The second is a remote sensing example to show its operation in unsupervised classification.

Slide 2.20.2 We now implement the k means algorithm using the two-dimensional data set shown here; we will see this data set a couple of times in these lectures.

Looking at the pixel locations in this data it seems there are two or possibly three clusters. In practice that might not be obvious, so a guideline is needed in terms of the initial choice of how many clusters to find.

Because we can merge clusters later, it is good to estimate on the high side, recognizing however that the more clusters there are the longer the algorithm is likely to take to converge. A guideline, which has been used in remote sensing, is to estimate the number of information (ground cover) classes in a scene and then search for 2 to 3 times that number of clusters.

Slide 2.20.3 The choice of the initial cluster *positions* is important because it can influence the speed of convergence and the ultimate set of clusters. It is important that the initial set be spread well across the data.

Several guidelines are available. One is that the initial cluster centres are spaced uniformly along the multi-dimensional diagonal of the spectral space. That is a line from the origin to the point of maximum brightness on each axis.

Better still we could choose the multi-dimensional diagonal that joins the actual spectral extremities of the data; that requires a bit of pre-processing to find the lower and upper spectral limits in each band, but that is quite straightforward.

Another approach is to space the initial cluster centres uniformly along the first principal component of the data. That will work well for highly correlated data sets but might be less useful if the data sets show little correlation.

Slide 2.20.4 Diagrammatically this slide shows the evolution of the k means method, iteration by iteration, on our small data set. Here we are searching for two clusters

The bottom row shows four iterative assignments of the pixels to the clusters, along with the corresponding SSE values. After the fourth iteration (assignment) there is no further migration of the pixels between the clusters. Note the pixels which change clusters in the first two steps.

The top right diagram shows how the means migrate with iteration. That tells us why the algorithm is sometimes called the method of migrating means.

Slide 2.20.5 The Isodata algorithm is a variation of the simple k means approach. It adds two or three further possible steps, as outlined in this slide.

- We can check to see whether any clusters contain so few pixels as to be meaningless. If the statistics of clusters are important, say for use in a later maximum likelihood classification, then poor estimates of those statistics will be obtained if the clusters do not contain a sufficient number of members.
- We can see whether any pairs of clusters are so close that they should be merged. In module 3 we will look at similarity measures for use in classification. They will give an indication of whether classes (and clusters) are too similar spectrally as to be useful.
- We can check whether some clusters are so elongated in some spectral dimensions that it would be sensible to split them. Elongated clusters are not necessarily a problem, but if they are, then comparison of the standard deviations of the clusters along each spectral dimension will help reveal their elongated nature.

Slide 2.20.6 We now look at the application of clustering to unsupervised classification, using a five channel data set of an image recorded by the HyMap sensor near the city of Perth in Western Australia in January 2010.

The table shows the where the five bands are located in the spectrum. From our knowledge of spectral reflectance characteristics, that choice seems sensible in being able to differentiate among the common cover types we expect to see.

The image display uses just three of the bands, selected to give the standard “colour-IR” product, in which vegetation is emphasized in red.

Slide 2.20.7 The remote sensing image analysis package called MultiSpec was used for this exercise. Six clusters were specified, with no provisions included to check close or elongated clusters. However, if any cluster was found to have fewer than 125 pixels it would be eliminated.

Once clustering was complete the unsupervised cluster map shown was produced.

The clusters, represented by different colours, follow the visual patterns of the classes in the image. It is easy to associate the brown and orange clusters with highways, road pavements and bare regions; the yellows with buildings; and the shades of green with various types of vegetation. Clearly, the dark blue cluster is water.

We strengthen this interpretation further in the next slide.

Slide 2.20.8 In the table here we see the mean vectors of the final set of clusters which, with the spatial clues in the map itself, allow us to associate the cluster colours with ground cover classes, as indicated in the key to the cluster map (now a thematic map).

- Slide 2.20.9** Here we plot the cluster means by wavelength and see that they follow the spectral reflectance curves of the ground cover class label assigned to the cluster. This is further information that has been used to identify the clusters. In so far as is possible, it is always good to look at ancillary representations such as this to help understand the identity of the clusters that have been found by the algorithm.
- Slide 2.20.10** It is instructive to see where the cluster centres lie in spectral space. While we can't envisage the five-dimensional space, we can look at two-dimensional scatterplots using two of the bands most significant to vegetation—the near IR and visible red bands. As seen here, the final cluster centres represent *well* the scatter of pixel vectors.
- Slide 2.20.11** Here we summarise the essential elements of the Isodata algorithm and how clustering is used for unsupervised classification.
- Slide 2.20.12** The first question here should allow you to develop a feeling for the importance of the placement of the initial cluster centres.

Lecture 21. Other clustering methods

Slide 2.21.1 In this lecture we summarise other methods for clustering and illustrate one of them so that we can compare its performance with that of k means clustering.

Slide 2.21.2 Although the k means algorithm is one of the most widely-used methods for clustering, there are other approaches that have been used with remote sensing data.

In the next lecture we will explore a recent clustering method that has been applied to hyperspectral data and to big data sets. But here we will look at another long-standing technique so we can see its performance relative to the k means algorithm. By doing so, we will demonstrate that the results of clustering are not unique, a fact that the user needs to be aware of, and handle carefully, when undertaking an unsupervised classification.

Other methods that have been used in remote sensing are

- Hierarchical clustering (which when applied to the first example of the last lecture tends to lead to three and not two clusters). It has been used as the basis for clustering in some big data applications. For details see J.A. Richards, *Remote Sensing Digital Image Analysis*, Springer, Berlin, 2013.
- Histogram peak selection, mountain climbing or density maximum selection
- Single pass clustering, which is the method we will now examine.

Slide 2.21.3 The single pass method is an old technique and had its origins when remote sensing imagery was supplied on sequentially accessible storage media like magnetic tape, with which iteration would be a particularly time-consuming process because of the need to read and rewind the tape. Despite its genesis, it is still sometimes used because of its simplicity and speed.

It assumes that the data is arranged in the usual row and column format. If the image is very large, a random sample is taken of the pixels and the results arranged again by row and column.

Slide 2.21.4 The algorithm proceeds in the following manner:

The first row is used to obtain an initial set of cluster centres in the following manner:

- The first sample is used as the centre of the first cluster
- If the second sample is further away from the first *by more than* a user-specified critical distance, then it is used to start a second cluster. Otherwise the two samples are assumed to be from the same cluster, in which case they are merged, and their mean computed.
- This process is applied to all samples (pixels) in the first row.
- At the end of the first row the multidimensional standard deviations of the clusters generated are produced for use in the later rows.

Each sample in the second and subsequent rows is checked to see if it lies within a user-specified number of standard deviations of one of the clusters from the first row. If it does it is added to that cluster, and the cluster statistics are recalculated. Otherwise it is used to start a new cluster and allocated a nominal standard deviation in each band.

Slide 2.21.5 In this slide we show the single pass method diagrammatically. The left-hand diagram shows how the first four samples are treated in this particular illustration. Only samples 2 and 3 are close enough to be merged. Clearly sample 2 was too far away from sample 1 and was used to start a new cluster. Also, sample 4, being too far away from the two existing clusters, is used to start another cluster.

The right-hand diagram shows how sample $n + 1$ falls within the prescribed number of standard deviations of cluster 2 and becomes part of that cluster, whereas sample n was too far away and is used to initiate another separate cluster.

Slide 2.21.6 The single pass method is fast and does not require the number of clusters to be pre-specified. It does, however, require the user to specify two parameters—the critical distance used with the first line of samples, and the standard deviation multiplier used in the remaining lines. Also, since it initiates clustering on the first line of samples, it is biased by the samples in that line; there is no way to moderate that choice.

Variations on the single pass algorithm exist. Some let the user specify the actual initial cluster centres, while others use a critical distance measure for all rows. The MultiSpec package operates that way.

Slide 2.21.7 We are now going to apply the single pass algorithm to the data set we treated in the last lecture with the k means method. The MultiSpec package was again used for this. It does not use the standard deviation method for the second and subsequent lines but applies another critical distance.

The critical distances used were 2500 and 2800 respectively for the first and subsequent lines of data. The numbers seem large, but this sensor has 16 bit radiometric resolution.

As noted in the previous slides, the algorithm uses the first line of pixels (or samples if the image is large) to initiate the cluster centres. In this case the first line is actually the right-hand column of pixels in the displayed image seen in next slide because, after clustering, the image and cluster maps were rotated 90 degrees clockwise to bring them into a north-south orientation.

- Slide 2.21.8** Here we see the results of the application of the single pass method to the image we analysed earlier. As with the k means algorithm, we can see that the clusters, represented by different colours, follow the visual patterns of the classes in the image. The colours here are different from before, and this time there is a confusion between roads and trees.
- Slide 2.21.9** In this slide we see the cluster centres created by the single pass algorithm. How do they compare to the clusters generated by the k means method? Well let's see...
- Slide 2.21.10** We compare the results of the two algorithms using bi-spectral plots, as shown here.
Note that the sparse vegetation, water and building classes are about the same for both algorithms, whereas the two approaches have picked up different combinations of bare surfaces, road and trees.
In practice, clustering may need to be refined by re-running the algorithm with different sets of parameters until a cluster set is obtained that matches the information classes of interest.
- Slide 2.21.11** Here we summarise the essential elements of the single pass algorithm and the fact that unique results are unlikely to occur.
- Slide 2.21.12** The third question is particularly important. Often in remote sensing we have the notion that the pixels tend to clump into groups that align with ground cover classes. That is not often the case. Instead, the spectral domain can look like a continuum with a few density maxima (clusters) associated with definite classes like water. We will have more to say about that in module 3.

Lecture 22. Clustering “big data”

Slide 2.22.1 In this lecture we look at how to do clustering with very big data sets, including hyperspectral images.

Slide 2.22.2 We are now in the era of “big data,” particularly with the recording and storage of many high-volume image data sets. In 2014 NASA was managing more than 9PB (petabyte) of data with about 6.4TB (terabyte) a day being added. (A terabyte is 10^{12} byte; a petabyte is 10^{15} byte). This accounts for an unbelievably large number of images.

How do clustering techniques cope with such large amounts of data?

If we want to apply clustering techniques to large images for unsupervised classification or use clustering to extract information from archived data sets—a process called data mining—then the methods for clustering we developed in the last couple of lectures are limited. In this lecture we will look at a recent clustering approach that is suitable for big data sets. There are others as well. But the one we look at now illustrates the types of method now being explored for use on so-called “big data.”

Slide 2.22.3 Just before doing that consider the time demand of the k means algorithm.

For P pixels, C clusters and I iterations, the k means algorithm requires PCI distance calculations. For N bands, the distance calculations involve N multiplications each, giving a total of $PCIN$ multiplications to complete a k means clustering exercise.

For a 1000x1000 image segment, involving 200 bands and searching for 15 clusters, if 100 iterations were required then 30×10^{10} multiplications are needed! How can we devise an approach to clustering that is much faster, and is able to cope effectively with large images?

Slide 2.22.4 In searching for an improved technique, the k means technique should not be abandoned. Its simplicity means it is still used with big data sets. To make it more suitable, though, there are several alternatives that should be examined.

- The simplest is to use a more powerful computer, but from an operational point of view it is important to note that most remote sensing practitioners would want to use readily available and not specialised computer hardware
- A better method for initiating the cluster centres might be found that helps speed up convergence by reducing the number of iterations needed; several methods have been suggested for that.
- A multi-processor (multi-core) machine could be used to speed up the computation by taking advantage parallel calculations; however, steps need to be taken to parallelise the k means algorithm which, because of its iterative nature, requires some innovative modifications
- A more efficient version of the k means algorithm might be possible; we will examine one such technique here. It speeds up significantly the time required to undertake clustering and to allocate a pixel to a cluster class. In a sense this is a particular case of the third dot point.

Slide 2.22.5 The method we will look at for fast clustering is called K trees. We met decision trees in the context of the support vector machine, but now we want to look at them more generally.

We start with some nomenclature.

- Trees consist of nodes, linked by branches.
- The uppermost node is called the root, and the lowermost nodes are called leaf nodes.
- In between there are internal nodes.

The nodes are arranged in layers, as shown. Progression of a pixel down the tree is based on decisions at the nodes; those decisions direct the pixel into one of the available branches.

Slide 2.22.6 In the K trees algorithm, we allocate leaf nodes to the clusters that we are trying to find, both in number and position *in the spectral space*, although we don't know how many there will be beforehand.

Some authors use the leaf nodes to represent the individual pixels, with the clusters being the internal nodes in the layer directly above. That does not help in developing the algorithm and just adds an additional, unnecessary complication.

The K trees algorithm has one parameter that the user has to specify beforehand—it is called the tree order, which specifies that maximum number of pixels in a cluster and, as we will see in the following slides, the maximum population of any node in the tree.

Slide 2.22.7 Full details of the algorithm will be found in this paper by Geva. It is a little hard to understand in the remote sensing context since it is written in the language of computer science; so we will develop the algorithm by example, using a simple two-dimensional set of data, *and* using remote sensing terminology.

- Slide 2.22.8** We will use the set of 8 samples shown here in vector and diagram form, and choose a tree order of 3. Specification of the order controls the structure of the tree, as we will see.
- Slide 2.22.9** The tree starts with a single root node and a single leaf node. We then feed in the first sample, say c . This is also called **insertion**. Since we have no other information, it simply flows down to the leaf node, as does the second sample a , as shown on the right-hand side of the slide.
- We use black letters to indicate samples of current interest and red letters to indicate samples which have already been fed into the tree.
- Slide 2.22.10** A third sample, say g , can be accommodated, but it fills the leaf node, since we have specified a tree order of 3.
- A fourth sample, say d , cannot be accommodated in the current tree because the leaf node cannot contain more than 3 samples by design. That leaf node has to be split.
- The K trees algorithm does the split by doing a k means clustering of the four samples, as on the next slide.
- Slide 2.22.11** The k means clustering in the K trees approach always looks for just two classes—so that the over-full leaf node is split into two new leaf nodes.
- In this example the vectors a, b, c, g have to be allocated to 2 clusters. We show that process in this slide. Because of the initial choice of cluster centres, the algorithm converges in one iteration for this very simple example.
- When complete, the two clusters have the mean vectors indicated on the bottom of this slide.
- Slide 2.22.12** The mean vectors from the clustering step of the previous slide become the attributes (members) of the root node, and the internal leaf is split into the two clusters, as shown in the left-hand diagram.
- The tree now has capacity to absorb more pixels, so a 5th sample f can be inserted as seen on the right-hand side of the slide. That pixel vector is checked against the two mean vectors in the root node. It is closest to $\mathbf{m}_{cd}$ so it is allocated to the left-hand leaf node.
- Slide 2.22.13** As in the previous slide, when pixel b is inserted it is checked against the mean vectors held in the root node, and is found to be closer to $\mathbf{m}_{ag}$, as a result of which it is allocated to the right hand leaf node. The mean $\mathbf{m}_{abg}$ is calculated, and used in place of $\mathbf{m}_{ag}$ in the root node
- But when we try to insert the 7th sample h the capacity of the left-hand leaf node is exceeded and that node has to be split, again using the k means algorithm.

- Slide 2.22.14** We are now looking to separate the pixels c, d, f, h into two clusters using the k means approach. Rather than go through the exercise we assume, for simplicity, that the solution shown in the diagram has been found. The means of the new clusters are as shown. This leads to the tree now having three leaf nodes as seen on the next slide.
- Slide 2.22.15** The root node now contains three mean vectors and has reached its capacity. If any further leaf nodes are added, then the root will have to be split.
- Slide 2.22.16** Now consider the insertion of the final pattern e to the tree. When entering the root node, it is seen to be closest to the $\mathbf{m}_{abg}$ mean, so that it should be placed in the bottom right hand leaf as indicated. But since that exceeds that leaf's capacity, the leaf has to be broken into two separate leaves, again using k means clustering, as on the next slide.
- Slide 2.22.17** Again, for simplicity we assume that the k means algorithm has found the clusters illustrated in the diagram to the left, with the mean vectors indicated. The tree now has four leaf nodes as shown on the next slide.
- Slide 2.22.18** But the root node now needs to be split into two, because its capacity has been exceeded. The two new nodes resulting from the split will be internal nodes; and a new root node is created.
We split the root node using the k means approach, but the elements now to be clustered are the mean vectors stored in the root node.
- Slide 2.22.19** Again, assume the results of the k means clustering are as shown here. The new means $\mathbf{m}_{cdfh}$ and $\mathbf{m}_{abeg}$ are now computed.
- Slide 2.22.20** Since all pixels have now been fed into the tree, we have its final version, as seen here.
It has three layers, with two internal nodes and four leaf nodes. Any pixel vector fed into the top of the tree will make its way down to one of the clusters via the decisions (i.e. distance comparisons) at the root and internal nodes.
For example, the vector $\begin{bmatrix} 3.0 \\ 3.0 \end{bmatrix}$ will flow through as shown by the dotted green line, into the cluster c, h .
- Slide 2.22.21** Apart from “*how well do they cluster?*” there are two things we want to know about clustering algorithms. First, how long does it take to build the tree and, secondly, especially with unsupervised classification in mind, how quick is it at allocating unseen data to a cluster?
If we look at the speed of allocation first, we can do so by counting the number of distance comparisons. In the simple case here, both the K trees and the equivalent k means approach require the same number of comparisons. But what about with bigger data sets?

Slide 2.22.22 If we take the simplest case of each node in the K tree requiring two distance comparisons, the number of comparisons increases by 2 for each new layer added, which in this case also doubles the number of clusters.

By contrast, the number of distance comparisons for the k means algorithm goes up as powers of 2. So, for larger numbers of clusters the K trees algorithm is much faster when allocating an unseen sample to an existing cluster.

Slide 2.22.23 Getting a meaningful comparison of the times to build the K tree and the k means approach is not straightforward. We can make comments on the number of nodes to be built and the checks within them, but the complexity introduced by the effect of different tree orders makes meaningful theoretical comparisons difficult. So, we will use an example, instead, as in the next slide.

Slide 2.22.24 This example is taken from the paper cited.

The data to be clustered consisted of vector samples from a three dimensional normal distribution, with zero mean and unity variance. A range of sample sizes was used, starting at 1000 and progressing to 256,000 by successive doublings. The K tree order was chosen as $m = 50$.

For comparison a k means clustering was run on the same set of samples. It was initiated with the same number of clusters as found by the K trees approach on the same data set.

Ten sets of each test were done, with the results averaged.

Slide 2.22.25 Here we see the comparison, which is quite compelling. For a given number of samples to be clustered, the K trees approach is much, much faster than the k means algorithm, as seen.

As always, there is a trade-off. Geva (the author) found that the K trees clustering was not quite as accurate as the k means approach. Given that k means is an iterative procedure, which involves all pixels at all stages, while K trees is single pass per sample and segments the data space during learning by its branching structure, that is not surprising.

But in general, Geva found the difference not to be a problem in most practical situations, especially given the speed benefit of K trees.

Slide 2.22.26 Another significant factor in favour of the K trees approach is that it can be adapted to run on multi-core processors, and not to require all samples to be held in core memory during clustering. Those additional developments are new and contained in the paper by Woodley and others, cited in this slide.

Slide 2.22.27 We are now at the end of our lectures on clustering. It would be good to compare this summary with those of the previous three lectures in order to reinforce overall the important aspects of clustering, especially when used as a tool for unsupervised classification.

Slide 2.22.28 The first two questions here are important to the development of classification methodologies, which we will pursue in module 3. The last question highlights a particular benefit of clustering with K trees.